

**Universidad Tecnológica de La Habana
“José Antonio Echeverría”**

Facultad de Ingeniería Informática



**DESARROLLO DE UN MÓDULO DE SEGURIDAD POR
SOFTWARE PARA LA GESTIÓN DE CLAVES
CRIPTOGRÁFICAS Y FIRMA DIGITAL**

Trabajo para optar por el título de Ingeniería Informática

Autor: Andy Martínez-Aparicio González

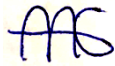
Tutores: Dr. C. Lisandra Bravo Ilisástigui
Ing. Rubén Sánchez Rivero

La Habana, Junio 2026

Declaratoria de autoría

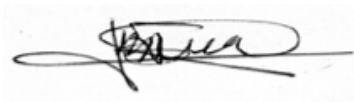
Declaro que soy el único autor de este trabajo y autorizo a la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana para que haga el uso que estime pertinente con este trabajo.

Para que así conste, firmo la presente a los 19 días del mes de junio de 2026.



Firma del autor

Andy Martínez-Aparicio González



Firma del tutor

Dr. C. Lisandra Bravo Ilisástigui



Firma del tutor

Ing. Rubén Sánchez Rivero

AGRADECIMIENTOS

A mi madre, por ser el pilar fundamental sin el cual este trabajo no habría sido posible. Por su ayuda incondicional, sus consejos siempre acertados, su habilidad para fijarse en los detalles y su carácter meticuloso, y por acompañarme despierta hasta la madrugada en tantas noches de este proceso.

A mi padre, por estar siempre pendiente de mis necesidades, por sus constantes llamadas y por su preocupación permanente porque todo en mi vida marchara bien.

A mi abuela Carmen, por brindarme su cariño en todo momento y por confiar, desde que era niño, en que llegaría a convertirme en un buen ingeniero.

A mi hermano, por su preocupación constante, por acompañarme a lo largo de este camino y por celebrar conmigo cada logro alcanzado.

A mi novia Claudia, por acompañarme en todo este proceso, por brindarme su cariño y su apoyo al máximo cuando lo necesité, y por confiar siempre en que yo podía lograrlo.

A mis amigos, Daryll, Andy, Piedra, Roberto y Marcelo, porque gracias a ellos esta carrera estuvo llena de risas y momentos divertidos que la hicieron mucho más amena. Estoy convencido de que son de los mejores informáticos de esta universidad, un verdadero talento al que le espera un futuro brillante, y espero que el camino nos permita coincidir en algún proyecto profesional más adelante.

A mis compañeros de tesis, Adriana y Orly, por el trabajo conjunto y por la capacidad de convertir en risas cada contratiempo y dificultad que surgió en el proceso, lo que nos permitió seguir adelante hasta el final compartiendo tanto las frustraciones como los logros.

A mis tutores. A Lisandra, por contribuir a que este trabajo alcanzara un nivel más profesional, por recibirnos en su casa siempre que le fue posible y por compartir con nosotros las mejores prácticas y consejos a lo largo de toda la tesis. A Rubén, por su apoyo en los aspectos científicos y técnicos del trabajo.

A mis profesores, especialmente a aquellos que disfrutaban de enseñar, que se esfuerzan porque sus estudiantes comprendan verdaderamente cada contenido y que siempre estuvieron dispuestos a brindar su atención y apoyo a lo largo de la carrera.

A los trabajadores de la empresa INGENIUS, por recibirnos siempre con amabilidad y cariño durante el desarrollo de la investigación.

A mis compañeros de aula, con quienes compartí estos años de carrera.

Muchas gracias a todos por hacer esto posible.

DEDICATORIA

A mis padres, por quererme y apoyarme siempre.

RESUMEN

La creciente utilización de la firma digital en sectores como el gobierno electrónico, la banca y el comercio digital exige mecanismos que garanticen la protección de las claves criptográficas empleadas durante el proceso de firma. En el contexto cubano, la ausencia de una infraestructura especializada para la custodia centralizada de claves privadas limita la seguridad y la adopción de estos servicios. La presente investigación tuvo como objetivo desarrollar un módulo de seguridad por software para la protección y gestión de claves criptográficas, así como para la ejecución segura de operaciones de firma digital. Para ello, se estudiaron los fundamentos teóricos, normativos y tecnológicos relacionados con la seguridad informática, la firma digital, los módulos criptográficos y la gestión de claves. Además, se analizaron soluciones existentes y se identificaron limitaciones relacionadas con la dependencia de infraestructuras externas, la soberanía de los datos y la complejidad operativa. Como resultado, se diseñó e implementó un módulo de seguridad accesible a través de una interfaz de servicios y desplegable en entornos de ejecución aislados. El módulo incorpora generación interna de claves, protección de claves privadas, gestión de su ciclo de vida, operaciones de firma y verificación digital, control de acceso basado en roles con autenticación federada y registro de auditoría. La validación mediante pruebas unitarias, de integración y del sistema evidenció el cumplimiento de los requisitos definidos. La propuesta constituye una alternativa de bajo costo, adaptable al contexto nacional, para fortalecer la seguridad de los servicios de firma digital.

Palabras clave: módulo de seguridad por software, gestión de claves criptográficas, firma digital, protección de claves privadas

ABSTRACT

The increasing use of digital signatures in sectors such as e-government, banking, and digital commerce requires mechanisms that ensure the protection of the cryptographic keys used during the signing process. In the Cuban context, the absence of specialized infrastructure for the centralized custody of private keys limits both the security and adoption of these services. This research aimed to develop a software-based security module for the protection and management of cryptographic keys, as well as for the secure execution of digital signature operations.

To achieve this objective, the theoretical, regulatory, and technological foundations related to information security, digital signatures, cryptographic modules, and key management were studied. In addition, existing solutions were analyzed, revealing limitations associated with dependence on external infrastructures, data sovereignty concerns, and operational complexity.

As a result, a security module accessible through a service interface and deployable in isolated execution environments was designed and implemented. The module incorporates internal key generation, private key protection, key lifecycle management, digital signing and signature verification operations, role-based access control with federated authentication, and audit logging. Validation through unit, integration, and system testing demonstrated compliance with the established requirements. The proposed solution provides a low-cost alternative adaptable to the national context, contributing to strengthening the security of digital signature services.

Keywords: software-based security module, cryptographic key management, digital signature, private key protection

ÍNDICE DE CONTENIDO

INTRODUCCIÓN	1
CAPÍTULO I. MARCO TEÓRICO	5
1.1 Seguridad informática	5
1.1.1 Principios de confidencialidad, integridad y disponibilidad...	5
1.1.2 Criptografía y mecanismos de cifrado	6
1.1.3 Modelos y mecanismos de control de acceso	9
1.2 Firma electrónica y firma digital	10
1.2.1 Clasificación de la firma electrónica	11
1.2.2 Modelos de firma digital: desacoplada y embebida	12
1.3 Módulo criptográfico	13
1.3.1 Módulo de seguridad de hardware	13
1.3.2 Módulo de seguridad basado en software	14
1.4 Gestión del ciclo de vida de claves criptográficas	15
1.4.1 Ciclo de vida de claves criptográficas	15
1.4.2 Protocolo de interoperabilidad para la gestión de claves criptográficas	17
1.5 Estándares y normas aplicables	18
1.5.1 Marco regulatorio internacional	18
1.5.2 Marco regulatorio cubano	20
1.6 Estado del arte y selección de tecnologías	20
1.6.1 Análisis de soluciones existentes	21
1.6.2 Análisis comparativo e identificación de brechas	24
1.6.3 Selección de tecnologías para la solución propuesta	25
1.7 Conclusiones parciales	28
CAPÍTULO II. DISEÑO DEL MÓDULO DE SEGURIDAD BASADO EN SOFTWARE	30
2.1 Requisitos del módulo de seguridad basado en software	30
2.1.1 Requisitos funcionales	30
2.1.2 Requisitos no funcionales	32
2.2 Arquitectura del sistema	34
2.2.1 Patrón arquitectónico N-Capas	34
2.2.2 Patrones de diseño	37

2.3	Interfaz de servicios y control de acceso	39
2.4	Modelo de base de datos	42
2.5	Conclusiones parciales	44
CAPÍTULO III. IMPLEMENTACIÓN DEL MÓDULO DE SEGURIDAD		
POR SOFTWARE		46
3.1	Implementación del ciclo de vida de claves criptográficas	46
3.1.1	Generación de claves	46
3.1.2	Rotación de claves	46
3.1.3	Destrucción segura y zeroización	47
3.2	Protección de claves privadas en reposo.....	47
3.2.1	Arquitectura de clave de cifrado de claves	47
3.2.2	Algoritmo de cifrado	48
3.2.3	Flujo de cifrado y descifrado	48
3.3	Firma digital y verificación	50
3.3.1	Algoritmos criptográficos de firma y modelo de operación .	50
3.3.2	Verificación de firmas.....	52
3.4	Registro de auditoría	53
3.4.1	Registro de solo inserción	53
3.4.2	Operaciones auditadas	53
3.5	Modelo de despliegue.....	54
3.6	Conclusiones parciales	56
CAPÍTULO IV. VALIDACIÓN DE LA SOLUCIÓN		58
4.1	Pruebas unitarias	58
4.2	Pruebas de integración	66
4.3	Pruebas del sistema	73
4.4	Conclusiones parciales	76
CONCLUSIONES		78
RECOMENDACIONES		79
REFERENCIAS BIBLIOGRÁFICAS		80
GLOSARIO DE SIGLAS		86
GLOSARIO TÉRMINOS		87

ÍNDICE DE FIGURAS

Figura 1. Tríada CIA (Confidencialidad, Integridad y Disponibilidad)	6
Figura 2. Diagrama de casos de uso del sistema.....	32
Figura 3. Diagrama N-Capas.....	35
Figura 4. Diagrama de clases del patrón Facade (Fachada).....	37
Figura 5. Diagrama de clases del patrón Strategy (Estrategia)	38
Figura 6. Diagrama de clases del patrón <i>DTO</i> (Objeto de Transferencia de Datos).....	39
Figura 7. Diagrama de secuencia del flujo de autenticación y autorización	41
Figura 8. Diagrama del modelo físico de base de datos.....	43
Figura 9. Diagrama del proceso de cifrado.....	50
Figura 10. Diagrama del proceso de firma digital	52
Figura 11. Diagrama del modelo de despliegue	54

ÍNDICE DE TABLAS

Tabla 1. Comparación de soluciones de gestión de claves criptográficas	24
Tabla 2. Especificaciones de endpoints de la API REST	40
Tabla 3. Pruebas unitarias. Módulo de cifrado AES-256-GCM	60
Tabla 4. Pruebas unitarias. Módulo de firma digital, <i>RSA</i>	61
Tabla 5. Pruebas unitarias. Módulo de firma digital, <i>ECDSA</i>	61
Tabla 6. Pruebas unitarias. Módulo de firma digital, validaciones de negocio .	62
Tabla 7. Pruebas unitarias. Ciclo de vida de claves.....	63
Tabla 8. Resultados de las pruebas unitarias por iteración	65
Tabla 9. Prueba de integración. Arranque del contexto	68
Tabla 10. Pruebas de integración. Creación y consulta de claves.....	68
Tabla 11. Pruebas de integración. Firma y verificación	69
Tabla 12. Pruebas de integración. Ciclo de vida de claves.....	70
Tabla 13. Pruebas de integración. Control de acceso y seguridad	71
Tabla 14. Pruebas de integración. Registro de auditoría	71
Tabla 15. Resultados de las pruebas de integración por iteración.....	72
Tabla 16. Prueba de extremo a extremo. Cliente web Streamlit.....	74
Tabla 17. Resultados de las pruebas del sistema por iteración	76

INTRODUCCIÓN

La firma digital se entiende como un mecanismo criptográfico esencial para garantizar la autenticidad, integridad y no repudio de los documentos electrónicos. Su utilización se ha incrementado de manera sostenida en sectores como el gobierno electrónico, la banca, la salud y el comercio digital debido a la necesidad de establecer entornos de intercambio de información confiables y jurídicamente válidos [1, 2]. En consecuencia, la protección de las claves privadas empleadas durante el proceso de firma adquiere una relevancia crítica, puesto que la exposición de dichas claves compromete todas las firmas emitidas y afecta la confianza depositada en la infraestructura criptográfica que las sustenta [3].

Con el propósito de garantizar elevados niveles de seguridad en la custodia de claves criptográficas, la industria consolidó el empleo de los módulos de seguridad de hardware (*HSM*, por sus siglas en inglés, *Hardware Security Module*), dispositivos específicamente diseñados para ejecutar operaciones criptográficas dentro de entornos seguros y aislados [4]. En este tipo de soluciones, las claves privadas nunca abandonan el dispositivo. Este principio, conocido como no exposición de claves, es considerado uno de los fundamentos de las infraestructuras modernas de firma digital. Su importancia ha sido respaldada por estándares internacionales, como FIPS 140-3 del Instituto Nacional de Estándares y Tecnología (*NIST*, del inglés *National Institute of Standards and Technology*), vigente para la validación de módulos criptográficos [5]. Aunque los *HSM* proporcionan elevados niveles de protección, presentan limitaciones relacionadas con sus costos de adquisición, mantenimiento y despliegue, situación que dificulta su adopción en organizaciones con restricciones presupuestarias o infraestructuras tecnológicas limitadas [4, 6].

Como respuesta a esta problemática, surgieron los módulos de seguridad por software, soluciones que implementan mecanismos de protección criptográfica sobre servidores de propósito general [6]. Estas plataformas permiten generar las claves criptográficas internamente, almacenarlas cifradas, controlar el acceso mediante políticas basadas en roles y registrar las operaciones realizadas con fines de auditoría y trazabilidad. Aunque no alcanzan el nivel de blindaje físico de un *HSM* certificado, se consideran una alternativa técnicamente viable para

entornos que requieren garantizar la seguridad criptográfica sin disponer de hardware especializado [7]. De esta forma, los módulos de seguridad por software se han convertido en una opción de creciente relevancia dentro de las infraestructuras contemporáneas de firma digital.

En correspondencia con esta tendencia, la normativa vigente en Cuba establece requisitos específicos para la protección de las claves privadas utilizadas en los procesos de firma electrónica, conforme a marcos regulatorios internacionales de referencia [8]. En este contexto se encuentra la plataforma eFirma, desarrollada por la empresa INGENIUS, que ofrece servicios de firma digital mediante aplicaciones *web* y móviles. No obstante, el flujo actual de operación de la plataforma requiere que el usuario cargue manualmente un archivo contenedor *PKCS#12*, donde almacena la clave privada junto con el certificado digital, y proporcione la contraseña correspondiente durante cada operación de firma. Este procedimiento provoca que la clave privada se exponga temporalmente fuera del entorno controlado del usuario, incrementa la dependencia de una adecuada custodia del archivo criptográfico y amplifica los riesgos asociados a pérdida, duplicación o interceptación de credenciales [9]. Como consecuencia, la ausencia de un mecanismo centralizado y seguro de custodia de claves privadas limita la robustez del proceso de firma digital y dificulta la adopción masiva de este tipo de servicios en el país [1, 10].

La problemática descrita trasciende el caso específico de eFirma y se manifiesta en numerosos sistemas de firma digital que carecen de una infraestructura especializada para la gestión segura de claves criptográficas. Esta situación evidencia la necesidad de desarrollar soluciones capaces de administrar el ciclo de vida completo de las claves, desde su generación y almacenamiento hasta su destrucción segura, de forma tal que se garantice en todo momento la no exposición de las claves privadas [11]. A partir de este escenario se formula el siguiente **problema de investigación**: ¿cómo desarrollar un módulo de seguridad por software que garantice la protección y gestión de claves criptográficas, así como la ejecución segura de operaciones de firma digital?

En función del problema de investigación planteado, se define como **objeto de estudio** los módulos de seguridad por software y como **campo de acción** la gestión de claves criptográficas y firma digital mediante módulos de seguridad por software. Sobre esta base, el **objetivo general** del trabajo consiste en

desarrollar un módulo de seguridad por software que garantice la protección y gestión de claves criptográficas, así como la ejecución segura de operaciones de firma digital.

Para alcanzar este propósito se establecen los siguientes objetivos específicos y sus correspondientes tareas de investigación:

Objetivo específico 1: Identificar los conceptos, normativas y tecnologías asociados a los módulos de seguridad por software y la gestión de claves criptográficas.

- 1.1. Estudiar los conceptos relacionados con *HSM*, *HSM* por software, ciclo de vida de claves y semántica *KMIP*.
- 1.2. Revisar las normativas y estándares aplicables al desarrollo de módulos de seguridad criptográfica.
- 1.3. Analizar las soluciones existentes de gestión de claves y sus limitaciones en el contexto cubano.
- 1.4. Seleccionar las tecnologías a emplear en el desarrollo del sistema.

Objetivo específico 2: Diseñar el módulo de seguridad por software a partir de la definición de los requisitos, la arquitectura y el modelo de datos.

- 2.1. Definir los requisitos funcionales y no funcionales del módulo de seguridad por software.
- 2.2. Diseñar la arquitectura general, el modelo de datos y los flujos criptográficos del sistema.
- 2.3. Diseñar la *API REST* con mecanismos de control de acceso basados en roles.

Objetivo específico 3: Implementar el módulo de seguridad por software según el diseño establecido.

- 3.1. Desarrollar el motor criptográfico de gestión de claves con protección de claves privadas en reposo.
- 3.2. Implementar las operaciones de firma y verificación digital.
- 3.3. Integrar el módulo de auditoría para el registro inmutable de operaciones.

Objetivo específico 4: Validar el módulo desarrollado mediante pruebas unitarias, de integración y de sistema.

- 4.1. Realizar pruebas unitarias sobre los componentes criptográficos.
- 4.2. Ejecutar pruebas de integración sobre los flujos funcionales.

4.3. Realizar pruebas de sistema para verificar el comportamiento integral de la solución y el cumplimiento de los requisitos definidos.

4.4. Analizar los resultados obtenidos durante el proceso de validación.

El valor práctico de esta investigación radica en el desarrollo de un sistema informático desplegable mediante contenedores *Docker*. Dicho sistema implementa un módulo de seguridad por software accesible a través de una *API REST* para la gestión de claves criptográficas, así como para la ejecución de operaciones de firma y verificación digital. La solución propuesta permite generar las claves privadas internamente, almacenarlas cifradas en reposo, registrar de manera inmutable las operaciones realizadas y controlar el acceso mediante diferentes niveles de privilegio. De esta forma, el sistema proporciona una alternativa de infraestructura criptográfica de bajo costo, reproducible y adaptable a plataformas de firma digital que requieran custodia centralizada de claves privadas, sin disponer de hardware especializado.

El trabajo se estructura en cuatro capítulos que abordan de manera progresiva los fundamentos teóricos, metodológicos y prácticos de la investigación. El primer capítulo desarrolla los conceptos relacionados con la gestión de claves criptográficas, los módulos de seguridad de hardware y software, los algoritmos y estándares aplicables, así como el análisis de soluciones existentes y las tecnologías seleccionadas. El segundo capítulo presenta el diseño del sistema que incluye los requisitos, la arquitectura general, la *API REST* y el modelo de datos.

El tercer capítulo describe la implementación del módulo de seguridad por software, con énfasis en los mecanismos de gestión y protección de claves, el control de acceso y la auditoría. El cuarto capítulo presenta el proceso de validación de la solución, el análisis de los resultados obtenidos y la verificación del cumplimiento de los requisitos definidos. Finalmente, se exponen las conclusiones y recomendaciones derivadas de la investigación, junto con las referencias bibliográficas que sustentan el trabajo realizado.

CAPÍTULO I. MARCO TEÓRICO

En este capítulo se analizan los fundamentos conceptuales, normativos y tecnológicos que sustentan el desarrollo de un módulo de seguridad por software orientado a la gestión de claves criptográficas y a la firma digital. Se abordan los principios de la seguridad informática y los mecanismos de cifrado que la sustentan, así como los conceptos de firma electrónica y firma digital junto con sus modelos de implementación. Asimismo, se analiza el ecosistema de los módulos de seguridad basados en hardware y software, la gestión del ciclo de vida de las claves y el estándar que la formaliza. Además, se examinan las normativas nacionales e internacionales aplicables, junto con las soluciones existentes en el estado del arte y la selección de tecnologías derivada de su análisis. El capítulo finaliza con las conclusiones parciales que recogen los principales resultados del análisis desarrollado.

1.1 Seguridad informática

La seguridad informática comprende principios, mecanismos y procedimientos orientados a proteger la confidencialidad, integridad y disponibilidad de la información frente a accesos no autorizados, alteraciones indebidas y amenazas cibernéticas. En los entornos digitales contemporáneos, caracterizados por la interconexión de servicios, la computación en la nube y el intercambio masivo de datos, esta disciplina se ha convertido en un componente estratégico para garantizar la continuidad operativa y la protección de los activos digitales [2, 12, 13].

1.1.1 Principios de confidencialidad, integridad y disponibilidad

El modelo más extendido para describir los objetivos de seguridad de un sistema de información es la tríada de confidencialidad, integridad y disponibilidad (*CIA*, por sus siglas en inglés) [2, 14, 15]. En la Figura 1 se representa la relación entre estas propiedades.

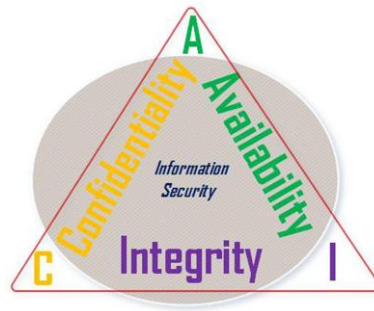


Figura 1. Tríada CIA (Confidencialidad, Integridad y Disponibilidad) [15]

La confidencialidad se entiende como el conjunto de restricciones sobre el uso y el almacenamiento de diversos tipos de datos [2]. En este sentido, garantiza que el acceso a la información y a los informes de datos se otorgue únicamente al usuario autorizado. Dicho acceso está sujeto a la necesidad de acceder a la información [15]. En este caso, el mecanismo técnico fundamental es el cifrado, el cual consiste en transformar la información en un formato ilegible para quien no posea la clave correspondiente, lo que impide el acceso no autorizado incluso si el medio de almacenamiento o transmisión se ve comprometido [16].

La integridad es la garantía de que los datos no han sido manipulados [2]. Esta se preserva cuando la información está bien protegida. Los datos cifrados solo pueden ser modificados por el usuario autorizado durante el procesamiento, la transmisión y el almacenamiento [15].

La disponibilidad de los datos es fundamental y establece que los recursos del sistema deben ser accesibles cuando los usuarios autorizados los requieran [2]. La seguridad de la información y la autenticación son vitales para la protección de estos. Sin embargo, también resulta imprescindible que los datos estén disponibles en el momento requerido, incluso en situaciones de emergencia o de carácter crítico [15].

1.1.2 Criptografía y mecanismos de cifrado

La clave criptográfica es el elemento fundamental de un sistema criptográfico que se utiliza para realizar operaciones de cifrado, descifrado, autenticación y protección de la información. Es un parámetro secreto que controla la transformación matemática aplicada a la información en dichas operaciones. Su longitud, expresada en bits, determina el espacio de claves posibles y, por tanto, la resistencia del sistema frente a ataques de fuerza bruta: a mayor longitud, mayor dificultad computacional para obtener la clave sin conocerla [11]. Del

mismo modo, la seguridad de los sistemas criptográficos modernos basados en claves depende de que estas sean generadas con suficiente aleatoriedad, mantenidas en secreto y destruidas de forma segura cuando dejan de ser necesarias [11, 17].

El cifrado es el proceso mediante el cual se transforma información legible en un formato ilegible, denominado texto cifrado, de modo que solo quien posea la clave adecuada pueda revertir la transformación y recuperar la información original [16]. Según la forma en que se gestionan las claves involucradas, el cifrado se clasifica en dos grandes familias: el cifrado simétrico y el cifrado asimétrico.

Cifrado simétrico

En el cifrado simétrico, la misma clave se emplea tanto para cifrar como para descifrar la información. Las dos partes que intervienen en una comunicación deben conocer y custodiar dicha clave de forma secreta. Su principal ventaja radica en la eficiencia computacional. Los algoritmos simétricos son significativamente más rápidos que sus equivalentes asimétricos y resultan adecuados para cifrar grandes volúmenes de datos [16].

Su limitación fundamental es el problema de distribución de la clave. Para que dos partes puedan comunicarse de forma segura, deben acordar previamente una clave compartida a través de un canal seguro. Esto, en entornos con múltiples participantes, introduce una complejidad que crece con el número de usuarios.

Los algoritmos simétricos se aplican habitualmente en el cifrado de datos en reposo (por ejemplo, almacenamiento cifrado de archivos o bases de datos) y en el cifrado de datos en tránsito dentro de canales ya establecidos [18].

Cifrado asimétrico

El cifrado asimétrico, también denominado criptografía de clave pública, emplea un par de claves matemáticamente relacionadas: una clave pública y una clave privada. En este esquema lo que se cifra con una de ellas solo puede descifrarse con la otra. La clave pública puede distribuirse libremente, mientras que la clave privada debe permanecer en exclusiva posesión de su propietario y no debe ser compartida ni transmitida bajo ninguna circunstancia [19].

Este esquema solucionó el problema de distribución inherente al cifrado simétrico, ya que, para que una parte envíe información cifrada a otra, basta con

conocer su clave pública, sin necesidad de establecer previamente un canal seguro para acordar un secreto compartido.

La seguridad del cifrado asimétrico descansa en la dificultad computacional de resolver determinados problemas matemáticos, como la factorización de números enteros grandes o el logaritmo discreto en curvas elípticas [19]. Esta complejidad hace que, con los parámetros adecuados, sea computacionalmente inviable deducir la clave privada a partir de la clave pública en un tiempo razonable.

La clave privada constituye uno de los activos más críticos en cualquier sistema criptográfico asimétrico. Su compromiso invalida toda la información cifrada destinada a su propietario [10].

Funciones *hash* criptográficas

Una función *hash* criptográfica es una transformación matemática que convierte una entrada de longitud arbitraria en una salida de longitud fija, denominada resumen criptográfico. Esta transformación es determinista, por lo que una misma entrada produce siempre el mismo resumen, y su cálculo resulta computacionalmente eficiente [11, 20].

Las funciones *hash* criptográficas se distinguen de las funciones *hash* convencionales por satisfacer propiedades de seguridad específicas. La resistencia a la preimagen establece que, dado un resumen h , resulta computacionalmente inviable encontrar una entrada m tal que $hash(m)=h$, lo que confiere a la función un carácter prácticamente irreversible. La resistencia a la segunda preimagen establece que, dada una entrada m_1 , resulta computacionalmente inviable encontrar una entrada distinta m_2 que produzca el mismo resumen. Por su parte, la resistencia a colisiones establece que resulta computacionalmente inviable encontrar cualquier par de entradas diferentes m_1 y m_2 que generen un mismo resumen [11, 20].

La resistencia a colisiones tiene una implicación directa en la verificación de integridad descrita en la sección anterior. Debido al efecto avalancha característico de estas funciones, una modificación del contenido, incluso mínima, produce un resumen significativamente diferente e impredecible, lo que permite detectar alteraciones sin necesidad de comparar el documento completo.

El estándar de referencia para las funciones *hash* criptográficas empleadas en sistemas de seguridad es la familia *SHA-2* (*Secure Hash Algorithm 2*), definida por el *NIST* en el estándar FIPS 180-4 [20]. Esta familia comprende varias variantes, entre las que destacan *SHA-256*, *SHA-384* y *SHA-512*. *SHA-256* genera resúmenes de 256 bits (32 bytes) y proporciona una resistencia a colisiones equivalente a 128 bits de seguridad; *SHA-384* genera resúmenes de 384 bits (48 bytes) y proporciona una resistencia equivalente a 192 bits; mientras que *SHA-512* genera resúmenes de 512 bits (64 bytes) y proporciona una resistencia equivalente a 256 bits [20]. Estas variantes satisfacen las propiedades de seguridad descritas y cuentan con amplio soporte en las plataformas criptográficas modernas.

1.1.3 Modelos y mecanismos de control de acceso

El concepto de control de acceso puede entenderse como el conjunto de mecanismos, políticas y modelos destinados a regular y restringir la interacción de los usuarios o entidades con los recursos de un sistema [21]. Su finalidad consiste en garantizar que únicamente los sujetos autorizados puedan realizar acciones específicas sobre la información.

De este modo, el control de acceso se considera un componente esencial de la seguridad informática, ya que asegura la confidencialidad, integridad y disponibilidad de los recursos mediante la aplicación de políticas que determinan quién puede acceder, qué puede hacer y bajo qué condiciones.

Bajo este enfoque, los modelos de control de acceso permiten definir las autorizaciones de manera estructurada y flexible, mediante la asignación de permisos en función de roles predefinidos o de características contextuales del sujeto, del objeto y del entorno. Dentro de este contexto, se destacan dos modelos predominantes: el control de acceso basado en roles y el basado en atributos [21].

En el control de acceso basado en roles (*RBAC*, del inglés *Role-Based Access Control*), los permisos se asignan a roles, y estos, a su vez, se asignan a los usuarios. Este modelo es adecuado cuando los patrones de autorización son estables y bien definidos. Por otra parte, en el control de acceso basado en atributos (*ABAC*, del inglés *Attribute-Based Access Control*), las decisiones de autorización se toman a partir de la evaluación de los atributos asociados al

sujeto, al recurso y al entorno. Este enfoque ofrece mayor expresividad a costa de mayor complejidad de administración [21, 22].

En este contexto, los mecanismos modernos emplean protocolos de autenticación federada para validar la identidad de las entidades que acceden al sistema. Estos mecanismos se basan en *OpenID Connect (OIDC)*, lo que permite delegar la gestión de identidades en proveedores externos mediante el uso de *tokens* seguros para la validación de usuarios y servicios [23]. Igualmente, *OIDC*, en combinación con *OAuth 2.0*, facilita la obtención de *tokens* de acceso en formato *JSON Web Token (JWT)*, los cuales incorporan información sobre la identidad y los privilegios de los usuarios [24].

1.2 Firma electrónica y firma digital

La necesidad de atribuir autoría e integridad a los documentos en entornos digitales dio origen al concepto de firma electrónica. En términos generales, una firma electrónica se define como cualquier dato en formato electrónico asociado a un documento que permite identificar a quien lo suscribe [10, 25]. Su propósito es equivalente al de la firma manuscrita en el entorno físico; es decir, vincular a una persona con el contenido de un documento de manera verificable. Sin embargo, esta definición abarca mecanismos de seguridad heterogéneos que van desde un simple nombre escrito al final de un correo electrónico hasta operaciones criptográficas de alta seguridad. Esta diversidad exige una clasificación que distinga los niveles de garantía que ofrece cada mecanismo.

Desde esta perspectiva, la firma digital es el resultado de una transformación criptográfica que, cuando se implementa correctamente, permite verificar la autenticidad del origen, la integridad de los datos y el no repudio del firmante. [26]. Constituye el mecanismo criptográfico que sustenta los niveles avanzado y cualificado de la firma electrónica. Su fundamento es el cifrado asimétrico, en el cual el firmante utiliza su clave privada para generar una firma sobre el resumen criptográfico del documento, de modo que se obtiene un valor único asociado al contenido que solo él pudo generar [10, 27]. Dicho valor puede ser verificado por cualquier receptor mediante la clave pública correspondiente, sin necesidad de acceder a la clave privada del firmante. Este proceso permite garantizar la integridad del documento, al evidenciar cualquier modificación posterior a la firma, así como la autenticidad del origen, al confirmar que la firma fue generada por el titular legítimo de la clave privada asociada [28].

En conjunto, este mecanismo proporciona tres propiedades esenciales en la seguridad de la información digital: autenticidad, no repudio e integridad [28, 29]. La autenticidad permite identificar de forma inequívoca al firmante; el no repudio impide que este niegue la autoría del documento debido al carácter exclusivo de su clave privada; y la integridad permite detectar cualquier alteración del contenido, por mínima que sea, a través de cambios en el resumen criptográfico [29].

Es importante distinguir ambos términos con precisión: la firma electrónica es el concepto jurídico que describe cualquier mecanismo de atribución de autoría en entornos digitales [25], mientras que la firma digital es el mecanismo técnico criptográfico específico que permite alcanzar los niveles más altos de seguridad dentro de la firma electrónica [27, 29]. Toda firma digital es una firma electrónica, pero no toda firma electrónica es una firma digital.

1.2.1 Clasificación de la firma electrónica

El marco regulatorio europeo *eIDAS* (*electronic IDentification, Authentication and trust Services*), Reglamento UE 910/2014, establece la clasificación de referencia internacional para los tipos de firma electrónica [25]. Aunque se trata de un reglamento de la Unión Europea, su modelo de clasificación es adoptado como referencia técnica en múltiples jurisdicciones fuera de ella, incluidas las normativas cubanas en materia de firma digital.

La firma electrónica simple no impone requisitos técnicos específicos sobre los mecanismos empleados ni sobre la custodia de las credenciales del firmante. En consecuencia, su nivel de garantía es el más bajo, ya que no asegura ni la identidad del firmante ni la integridad del documento frente a posibles alteraciones.

La firma electrónica avanzada eleva los requisitos de forma significativa. Debe estar vinculada al firmante de manera única, permitir su identificación, haber sido creada con datos que el firmante puede utilizar bajo su exclusivo control, y ser capaz de detectar cualquier modificación posterior del documento firmado [25]. Este nivel exige el uso de criptografía asimétrica, ya que solo mediante el uso de una clave privada bajo control exclusivo del firmante es posible cumplir simultáneamente los requisitos de unicidad, identificación y detección de alteraciones.

La firma electrónica cualificada representa el nivel de mayor valor jurídico, al tener efecto legal equivalente al de la firma manuscrita. Además de cumplir todos los requisitos de la firma avanzada, exige que la clave privada sea custodiada por un dispositivo de creación de firmas cualificado (QSCD, del inglés *Qualified Signature Creation Device*) [25, 30]. Un QSCD garantiza que la clave privada se genera en su interior, no lo abandona en ningún momento y está protegida frente al acceso no autorizado [10].

La progresión desde la firma simple hasta la cualificada refleja un incremento en los requisitos técnicos que cada nivel impone sobre el mecanismo de firma subyacente. Los niveles avanzado y cualificado no pueden satisfacerse mediante mecanismos arbitrarios, sino que requieren específicamente el uso de la firma digital basada en criptografía asimétrica.

1.2.2 Modelos de firma digital: desacoplada y embebida

La relación entre la firma digital y el documento firmado permite identificar dos modelos arquitectónicos fundamentales ampliamente descritos en la literatura criptográfica y en los estándares de firma digital: la firma desacoplada y la firma embebida. Estos modelos se diferencian en la forma en que se estructura, almacena y transmite la evidencia criptográfica respecto del contenido firmado. En ambos casos, se preserva la validez del proceso de verificación basado en criptografía de clave pública [31].

El modelo de firma desacoplada se caracteriza por la existencia de una firma digital independiente del documento original. En este esquema, el documento y la firma se reconocen como artefactos separados, aunque criptográficamente vinculados mediante el valor de verificación derivado del resumen criptográfico del documento. El resultado son dos artefactos independientes: el documento y la firma, que el cliente conserva y distribuye de forma separada. [7, 31]. Este modelo facilita la transmisión independiente de los objetos criptográficos, lo que resulta especialmente útil en escenarios donde el documento debe permanecer inalterado o debe distribuirse sin encapsulamiento adicional [10, 31].

Por otro lado, en el modelo de firma embebida, la firma se incrusta dentro del propio archivo del documento y forma un único artefacto que contiene tanto el contenido como la evidencia de integridad [31]. En estos casos, la estructura del archivo almacena simultáneamente los datos y la firma, lo que permite su verificación como una unidad coherente sin necesidad de artefactos externos.

Ambos modelos se fundamentan en los principios de la criptografía asimétrica y en las funciones de *hash* criptográfico, tal como se establece en las recomendaciones del *NIST* para firmas digitales (FIPS 186-5) [26]. En consecuencia, la diferencia entre ellos no radica en el mecanismo criptográfico subyacente, sino en la estructura de encapsulación, almacenamiento y distribución del resultado de la firma. Esta distinción resulta relevante desde el punto de vista de interoperabilidad, gestión documental y escenarios de verificación distribuida.

1.3 Módulo criptográfico

Un módulo criptográfico es un componente de hardware, software o una combinación de ambos, diseñado para ejecutar operaciones criptográficas y proteger material criptográfico sensible, especialmente claves criptográficas, dentro de un entorno controlado y seguro. Estos módulos implementan mecanismos de generación, almacenamiento, uso, rotación y destrucción de claves, así como operaciones de cifrado, descifrado, firma digital y autenticación, y garantizan la confidencialidad e integridad del material criptográfico frente a accesos no autorizados [32].

La certificación más relevante para los módulos de seguridad corresponde al estándar FIPS 140-3. Este estándar clasifica los módulos criptográficos en cuatro niveles de seguridad crecientes. El Nivel 1 establece los requisitos mínimos relacionados con algoritmos y funciones criptográficas. El Nivel 2 incorpora mecanismos de evidencia física de manipulación. El Nivel 3 exige resistencia activa frente a intrusiones y destrucción automática de claves ante intentos de acceso físico no autorizado. Finalmente, el Nivel 4 representa el mayor nivel de protección al incorporar mecanismos completos de detección de ataques bajo cualquier condición del entorno [5].

1.3.1 Módulo de seguridad de hardware

Un módulo de seguridad de hardware (*HSM*) es un dispositivo especializado que incorpora procesadores criptográficos dedicados y mecanismos seguros de almacenamiento de claves y material sensible. Este ha sido diseñado para proteger el material criptográfico y ejecutar operaciones de firma y cifrado en un entorno aislado y controlado [33, 34]. Su propiedad fundamental es la no exposición de la clave privada debido a que las operaciones criptográficas ocurren en el interior del módulo y el sistema externo recibe únicamente el

resultado. Este enfoque permite garantizar la custodia segura del material criptográfico y se corresponde directamente con los principios de protección y aislamiento requeridos para la gestión segura de claves.

Estos dispositivos incorporan mecanismos de resistencia a la manipulación (*tamper-resistant*) y detección de manipulación (*tamper-evident*). Ante cualquier intento de apertura o intrusión física, el dispositivo puede destruir automáticamente las claves criptográficas almacenadas [33, 35]. Esta propiedad garantiza que incluso un atacante con acceso físico al dispositivo no pueda extraer el material criptográfico protegido.

Los *HSM* utilizados en infraestructuras críticas, como autoridades de certificación y sistemas bancarios, operan habitualmente con certificaciones de Nivel 3 o superior [36]. A pesar de sus ventajas en materia de seguridad, los *HSM* presentan elevados costos de adquisición e implementación. Además, la adopción de dispositivos certificados bajo FIPS 140-3 Nivel 3 requiere infraestructura especializada, procesos de integración complejos y personal técnico capacitado para su administración y mantenimiento [5, 6, 34]. Estas exigencias dificultan su utilización en entornos con recursos limitados y favorecen el empleo de alternativas basadas en software.

1.3.2 Módulo de seguridad basado en software

El *HSM* por software es una implementación de las funciones de un *HSM* mediante código, sin la utilización de un componente de hardware dedicado. Este se despliega sobre servidores convencionales o en entornos virtualizados, y proporciona un conjunto de operaciones criptográficas entre las que se incluyen la generación interna de claves, la firma digital, la gestión del ciclo de vida de las claves y el registro de auditoría [6].

Su principal ventaja frente al *HSM* radica en su menor costo y en la facilidad de despliegue, ya que no requiere adquisición de hardware especializado. Además, puede implementarse mediante tecnologías estándar de contenedorización y escalarse horizontalmente según la demanda de los sistemas [6]. Sin embargo, presenta como limitación la imposibilidad de alcanzar niveles superiores de certificación FIPS 140-3 sin controles de seguridad física certificados o hardware de soporte, debido a que la protección frente a ataques físicos depende del entorno de ejecución que lo aloja [5].

Esta limitación puede mitigarse mediante controles compensatorios, entre los que se incluyen el cifrado del material criptográfico en reposo mediante una clave maestra no persistente en la base de datos, el uso de autenticación federada, la aplicación de controles de acceso estricto y la implementación de mecanismos de auditoría inmutable para todas las operaciones realizadas. En escenarios donde el modelo de amenaza excluye el acceso físico al servidor y el costo de un *HSM* no resulta viable, el *HSM* por software representa una alternativa técnicamente válida [7].

La elección entre el *HSM* y el *HSM* por software depende del modelo de amenaza, los requisitos normativos y las restricciones presupuestarias del proyecto [34]. El *HSM* resulta apropiado cuando se contemplan ataques físicos al servidor o se exige certificación FIPS 140-3 Nivel 3 o superior [5]. En contraste, el *HSM* por software representa una alternativa viable cuando no se considera el acceso físico como vector de amenaza y existen limitaciones para la adquisición de hardware certificado, siempre que se complemente con controles de seguridad adecuados [6, 7].

1.4 Gestión del ciclo de vida de claves criptográficas

La gestión del ciclo de vida de claves criptográficas puede definirse como el conjunto de políticas, procesos y mecanismos técnicos destinados a controlar las claves criptográficas desde su generación hasta su destrucción segura [17]. La administración de este ciclo se considera un componente esencial de la infraestructura de seguridad criptográfica, debido a que la seguridad de los sistemas de cifrado y firma digital depende no solo de los algoritmos empleados, sino también de la protección y gestión adecuada de las claves durante todo su periodo de validez [17]. Desde esta perspectiva, la gestión controlada y auditable del ciclo de vida de las claves representa una responsabilidad fundamental de los módulos de seguridad, tanto físicos como por software.

1.4.1 Ciclo de vida de claves criptográficas

Una clave criptográfica no se reconoce como un recurso estático ya que su utilidad y nivel de seguridad varían a lo largo del tiempo. Una clave activa puede requerir reemplazo debido a políticas de rotación, quedar comprometida como consecuencia de un incidente de seguridad o expirar al superar el período de validez previsto [11, 37].

El ciclo de vida de una clave criptográfica comprende el conjunto de estados por los que transita desde su creación hasta su destrucción definitiva, así como las operaciones permitidas en cada uno de ellos. La guía *NIST SP 800-57* formaliza este ciclo mediante una máquina de estados que contempla seis estados: preactivación (*pre-activation*), activo (*active*), suspendido (*suspended*), desactivado (*deactivated*), comprometido (*compromised*) y destruido (*destroyed*) [17]. El presente trabajo adopta un modelo simplificado de cuatro estados derivado de este ciclo, en el que los estados suspendido y comprometido se consolidan operativamente en el estado revocado, por resultar suficiente para los requisitos de seguridad y simplicidad operativa del módulo desarrollado.

Una clave en estado activo ha sido generada correctamente y se encuentra habilitada para realizar operaciones criptográficas durante su periodo de validez. Una clave revocada ha sido deshabilitada de forma explícita antes de su expiración natural, por ejemplo, como resultado de una rotación planificada o de la sospecha de compromiso. Aunque no puede utilizarse para nuevas operaciones criptográficas, las firmas generadas antes de la revocación permanecen verificables, lo que preserva la validez de los documentos previamente firmados. Por su parte, una clave expirada ha superado su periodo de validez y no puede emplearse para nuevas operaciones. Finalmente, una clave destruida ha sido eliminada de forma irreversible tanto en memoria como en almacenamiento persistente [17]. Tras este proceso, el material criptográfico deja de existir, aunque el registro de metadatos asociado a la clave se conserva con fines de auditoría y trazabilidad.

Las operaciones que componen este ciclo incluyen la generación interna de la clave dentro del módulo seguro, su activación para uso criptográfico, la rotación que produce una nueva versión activa y transfiere la anterior al estado revocado, así como la destrucción segura [17]. Un principio derivado de este modelo, recogido en el estándar FIPS 140-3 e ISO/IEC 19790, establece que las claves privadas deben generarse siempre dentro del módulo. Si una clave privada se genera fuera de este entorno y posteriormente es importada, no puede garantizarse que no haya sido expuesta durante su creación o traslado, lo que compromete la integridad de la cadena de custodia [5, 32].

1.4.2 Protocolo de interoperabilidad para la gestión de claves criptográficas

La estandarización del ciclo de vida de claves criptográficas plantea un problema de interoperabilidad: si cada fabricante o desarrollador de módulos de seguridad establece sus propias operaciones y estados, los sistemas que interactúan con ellos quedan vinculados a una implementación específica. Para resolver esta problemática, la organización *OASIS* (del inglés, *Organization for the Advancement of Structured Information Standards*) publicó el protocolo *KMIP* (del inglés *Key Management Interoperability Protocol*), un estándar internacional que establece un lenguaje común para la gestión de objetos criptográficos entre módulos de seguridad y las aplicaciones que los utilizan [38].

KMIP no define algoritmos criptográficos ni especifica la forma en que deben implementarse internamente las operaciones. En su lugar, propone un conjunto estandarizado de operaciones y tipos de objetos criptográficos que los módulos de seguridad compatibles deben exponer y gestionar. Entre dichos objetos se incluyen claves simétricas, pares de claves asimétricas y certificados digitales, así como los estados del ciclo de vida por los que puede transitar cada uno de ellos [38].

Las operaciones fundamentales definidas por *KMIP* incluyen *Create*, para generar un nuevo objeto criptográfico dentro del módulo; *Get*, para recuperar objetos o sus atributos; *Locate*, para realizar búsquedas de objetos por criterios; *Activate*, para habilitar objetos en estado operativo; *Revoke*, para desactivar estos mediante una causa declarada; *Destroy*, para eliminar el material criptográfico; *Sign*, para generar firmas digitales; y *Verify*, para validar las firmas mediante claves públicas correspondientes [38].

La adopción de *KMIP* como modelo de referencia para el diseño de interfaces de módulos de seguridad proporciona una ventaja significativa: las operaciones expuestas, los estados gestionados y las restricciones aplicadas en cada etapa del ciclo de vida quedan fundamentadas en un estándar internacional reconocido y no en implementaciones propietarias o criterios *ad hoc* [38]. En este sentido, el módulo desarrollado en el presente trabajo no implementa *KMIP* como protocolo de transporte binario (*wire protocol*), ya que su utilización requiere clientes especializados e incorpora una complejidad que no aporta beneficios en el contexto de integración previsto. En su lugar, se adopta *KMIP* como referencia

conceptual. Los nombres y la semántica de las operaciones (*Create, Get, Sign, Verify, Revoke, Destroy*), así como el modelo de estados del ciclo de vida y las restricciones operativas asociadas, se toman del estándar y se exponen mediante una *interfaz REST/JSON*.

Esta distinción implica que el sistema no es compatible con clientes *KMIP* nativos a nivel de protocolo, pero sí lo es a nivel semántico. Este enfoque es coherente con el adoptado por sistemas como *HashiCorp Vault*, que mantiene su motor *Transit* con semántica derivada de *KMIP* expuesta vía *REST*, y ofrece adicionalmente un módulo *KMIP* nativo separado para escenarios que requieren compatibilidad de protocolo [38-40].

1.5 Estándares y normas aplicables

El desarrollo de un módulo de seguridad orientado a la custodia de claves criptográficas y a la firma digital se enmarca en un contexto regulatorio conformado por estándares técnicos de reconocimiento internacional y disposiciones jurídicas nacionales. Los estándares técnicos establecen los principios, criterios, requisitos y mecanismos que estos sistemas deben satisfacer para garantizar niveles adecuados de seguridad y confiabilidad. De forma complementaria, la normativa nacional representa el contexto jurídico en el que la firma digital adquiere validez legal y establece las exigencias aplicables a los mecanismos criptográficos utilizados en el territorio cubano.

1.5.1 Marco regulatorio internacional

El marco regulatorio internacional se sustenta en un conjunto de estándares técnicos ampliamente reconocidos, emitidos por organismos de normalización y entidades de referencia en seguridad de la información, que establecen los principios para el diseño e implementación de módulos de seguridad y servicios de confianza digital.

FIPS 140-3. El estándar FIPS 140-3 del *NIST* [5], establece los requisitos de seguridad para la validación de módulos criptográficos utilizados en la protección de información sensible. Publicado como sucesor de FIPS 140-2, armoniza sus requisitos técnicos con la norma ISO/IEC 19790, lo que proporciona un marco internacionalmente reconocido para la evaluación de módulos criptográficos. El estándar mantiene cuatro niveles de seguridad y contempla requisitos relacionados con la gestión de claves, autenticación, integridad operacional, interfaces, servicios criptográficos y protección frente a intentos de manipulación.

ISO/IEC 19790 [32] establece los requisitos de seguridad que deben cumplir los módulos criptográficos utilizados para la protección de información sensible. Asimismo, define un marco de evaluación que abarca aspectos como la delimitación del módulo, la autenticación de roles, la gestión del ciclo de vida de las claves, la seguridad física y lógica, y la integridad operacional. En el contexto de la firma digital y la custodia de claves criptográficas, la norma exige que las operaciones críticas, entre ellas la generación, el almacenamiento y el uso de claves privadas, se ejecuten dentro de un entorno controlado que impida su exposición externa. Este requisito contribuye a garantizar la confidencialidad e integridad del material criptográfico. Además, la norma establece criterios para evaluar la resistencia de los módulos frente a intentos de manipulación y extracción no autorizada de claves.

NIST SP 800-57 [17], se reconoce como una de las principales guías para la gestión del ciclo de vida de claves criptográficas. Este documento establece las longitudes mínimas de clave asociadas a distintos niveles de seguridad, los algoritmos aprobados para diversos escenarios de uso, las recomendaciones de rotación periódica de claves y los procedimientos de destrucción segura. Recomienda la separación entre las claves de datos y las claves de protección de claves (*KEK*, del inglés *Key Encryption Key*).

NIST SP 800-131A [41], define las directrices para la transición y discontinuación progresiva de algoritmos criptográficos y longitudes de clave que dejan de cumplir los niveles de seguridad requeridos con el tiempo. En particular, establece que las claves *RSA* (*Rivest–Shamir–Adleman*) de 2048 bits dejarán de cumplir los niveles mínimos de seguridad a partir del año 2030. Esta directriz tiene implicaciones directas sobre los tamaños de clave que un módulo de seguridad debe soportar, así como aquellos cuyo uso debe restringirse, con el objetivo de preservar la vigencia y solidez criptográfica del material generado a largo plazo.

eIDAS. El Reglamento UE 910/2014, conocido como *eIDAS* [25], configura el marco europeo que regula los servicios electrónicos de confianza e incluye la firma electrónica. Aunque *eIDAS* no aplica directamente al ordenamiento jurídico cubano, se considera una referencia internacional en el ámbito de la firma electrónica y ha influido en normativas nacionales. Entre los requisitos técnicos establecidos para la firma electrónica avanzada se incluyen la custodia exclusiva

por el firmante de la clave privada, la generación interna del par de claves y la trazabilidad mediante registros de auditoría, principios que se consideran la base para el diseño de un módulo de seguridad por software [30].

1.5.2 Marco regulatorio cubano

La normativa cubana en materia de seguridad criptográfica y firma digital ha avanzado hacia la alineación con los estándares internacionales. Las disposiciones vigentes establecen requisitos concretos sobre los módulos que custodian claves privadas para la firma digital.

Resolución 23/2022 del Ministerio del Interior regula los servicios de firma digital en Cuba [8]. Establece que los módulos utilizados para la firma y el sello electrónico de personas jurídicas deben cumplir requisitos de alta seguridad, estar acreditados y garantizar que la clave privada permanezca bajo control exclusivo del sistema de seguridad, sin posibilidad de extracción. Esta resolución refuerza los principios de confidencialidad, integridad y control exclusivo del material criptográfico en los procesos de firma digital.

Decreto-Ley 79/2023 [42], dispone el desarrollo, certificación, comercialización, importación y uso de dispositivos y sistemas de protección criptográfica en Cuba. Abarca estos dispositivos de manera amplia, e incluye tanto soluciones de hardware como de software, siempre que realicen operaciones criptográficas o protejan información mediante técnicas seguras. Introduce además la necesidad de control estatal sobre la certificación de tecnologías criptográficas empleadas en infraestructuras críticas.

Decreto 106/2024 [43], reglamenta la aplicación del Decreto-Ley 79/2023 y detalla los procedimientos técnicos y administrativos para la certificación, homologación y control de dispositivos y servicios criptográficos. Establece criterios de seguridad alineados con estándares internacionales como ISO/IEC 19790 e ISO/IEC 27001, que asegura la coherencia entre la normativa nacional y las mejores prácticas internacionales. Además, regula las condiciones bajo las cuales los sistemas criptográficos, incluidos los basados en software, pueden ser evaluados y aprobados para su uso oficial.

1.6 Estado del arte y selección de tecnologías

Con el objetivo de comprender el estado actual del conocimiento en la temática de los módulos de seguridad, resulta necesario realizar un análisis crítico de la literatura existente. Este proceso permite examinar y evaluar las soluciones

propuestas en el ámbito científico y técnico, así como identificar los principales enfoques, metodologías, tendencias y resultados alcanzados en el área. De manera similar, facilita la identificación de brechas tecnológicas y limitaciones que justifican el desarrollo de nuevas soluciones.

De acuerdo con lo anterior, el estado del arte comprende el estudio de las tecnologías disponibles, su nivel de adecuación a los requisitos de seguridad y sus principales limitaciones. A partir de esta evaluación, se establecen las bases para fundamentar la propuesta de investigación y orientar el diseño de las tecnologías más adecuadas para su implementación.

1.6.1 Análisis de soluciones existentes

El estudio de las soluciones existentes representa una etapa fundamental para evaluar las características técnicas, arquitectónicas y operativas de las plataformas actuales de gestión de claves criptográficas y módulos de seguridad. Este análisis permite determinar el grado de cumplimiento de los requisitos asociados a la custodia segura de claves, la firma digital, el control de acceso, la auditoría y la interoperabilidad criptográfica, así como identificar limitaciones relacionadas con la dependencia de infraestructura propietaria, la soberanía de los datos, la complejidad operativa y la integración con entornos nacionales.

Entre las tecnologías evaluadas se encuentran las plataformas *AWS KMS* y *AWS CloudHSM*, *Azure Key Vault* y *Azure Managed HSM*, *Google Cloud KMS* y *Cloud HSM*, así como *HashiCorp Vault*, solución desplegable en infraestructura propia.

AWS KMS y AWS CloudHSM

AWS KMS (Amazon Web Services Key Management Service) es el servicio de gestión de claves del proveedor de nube con mayor cuota de mercado a nivel mundial. Permite crear, almacenar y controlar claves criptográficas utilizadas para el cifrado de datos y la ejecución operaciones de firma digital, disponibles mediante una *API REST*. El control de acceso se implementa a través de *AWS IAM (Identity and Access Management)*, con políticas granulares de permisos por clave y por operación. Además, todas las operaciones quedan registradas automáticamente en *AWS CloudTrail*, que proporciona auditoría centralizada e inmutable. Por su parte, *AWS CloudHSM* extiende el servicio mediante módulos de hardware dedicados a un único cliente, certificados bajo FIPS 140-2 Nivel 3, en los cuales las claves no abandonan el entorno hardware bajo ninguna circunstancia [44].

Desde el punto de vista técnico, *AWS KMS* demuestra la solidez del modelo de operaciones criptográficas expuestas vía *API REST* con control de acceso por políticas de identidad. Sin embargo, su aplicación presenta las mismas limitaciones que las soluciones anteriores: es una plataforma propietaria de *Amazon* que requiere conectividad permanente con la infraestructura *AWS*, no admite despliegue en servidores propios, y la soberanía de los datos queda comprometida al depender de centros de datos externos [44].

Azure Key Vault y Azure Managed HSM

Azure Key Vault es el servicio de gestión de claves de la plataforma *Microsoft Azure*. Ofrece almacenamiento de secretos, certificados y claves criptográficas, así como operaciones de firma, cifrado y rotaciones de claves expuestas mediante una *API REST*. El control de acceso se implementa mediante el modelo *RBAC* de *Azure Active Directory*, con roles granulares asignables a nivel de bóveda o de clave individual. Todas las operaciones quedan registradas en *Azure Monitor*, que proporciona auditoría centralizada e integración con sistemas de alerta. La variante *Azure Managed HSM* extiende el servicio mediante módulos de hardware dedicados de un único inquilino, certificados bajo FIPS 140-3 Nivel 3, en los cuales las claves no abandonan el entorno hardware bajo ninguna circunstancia [44].

Desde el punto de vista técnico, *Azure Key Vault* demuestra la viabilidad del patrón de *API REST* autenticada con *OAuth 2.0* y *RBAC* para operaciones criptográficas [44]. Sin embargo, su aplicación presenta limitaciones significativas: es una plataforma propietaria de *Microsoft* que requiere conectividad permanente con la infraestructura *Azure*, no admite despliegue en servidores propios, y la soberanía de los datos queda comprometida al depender de centros de datos externos.

Google Cloud KMS y Cloud HSM

Google Cloud KMS es el servicio equivalente de *Google Cloud Platform* para la gestión de claves criptográficas en una jerarquía de tres niveles: proyectos, anillos de claves (*Key Rings*) y claves criptográficas (*CryptoKeys*). Cada clave admite múltiples versiones con rotación automática configurable. El control de acceso utiliza el sistema *IAM de Google*, con roles especializados por tipo de operación. La auditoría es automática mediante *Cloud Audit Logs*. La variante

Cloud HSM ancla las claves en módulos físicos certificados bajo FIPS 140-3 Nivel 3 gestionados por *Google* [44].

El modelo de organización jerárquica de claves junto con el soporte de versiones de clave y rotación automática, ilustra patrones de diseño relevantes para la gestión del ciclo de vida [44]. La separación de roles por tipo de operación es coherente con el modelo *RBAC* requerido en correspondencia con los marcos normativos analizados. Sin embargo, *Google Cloud KMS* presenta limitaciones: dependencia de infraestructura propietaria externa, ausencia de modalidad de despliegue en infraestructura propia y restricciones de soberanía de datos.

HashiCorp Vault

HashiCorp Vault es una plataforma para la gestión de secretos y la ejecución de operaciones criptográficas. A diferencia de las soluciones anteriores, puede desplegarse en infraestructura propia sin dependencias externas [39, 45]. Su motor *Transit* implementa operaciones de firma, verificación, cifrado y descifrado sin exponer las claves a las aplicaciones cliente, conforme al principio de aislamiento del material criptográfico característico de los módulos de seguridad. Además, soporta autenticación federada mediante *OIDC* y otros mecanismos, y su módulo *KMIP* permite integración con sistemas que emplean dicho protocolo de forma nativa [38, 40]. Para entornos de alta seguridad, admite sellado automático con *HSM*.

No obstante, desde agosto de 2023, *HashiCorp Vault* dejó de distribuirse bajo la licencia *MPL 2.0* y adoptó la *Business Source License (BSL)*. A diferencia de las licencias reconocidas por la *Open Source Initiative (OSI)*, la *BSL* permite imponer restricciones sobre determinados usos comerciales y competitivos, por lo que no cumple la definición estándar de software de código abierto. En consecuencia, la solución se clasifica actualmente como *source-available*, ya que el código fuente permanece accesible al público, aunque su utilización se encuentra sujeta a condiciones establecidas por el proveedor.

Además, su arquitectura orientada a la gestión de secretos de infraestructura en entornos empresariales introduce una complejidad operativa considerable [39]. La configuración de políticas, motores y autenticadores requiere conocimiento especializado, y la infraestructura de operación necesaria, que incluye nodos de alta disponibilidad y gestión de sellos, supera los requisitos de un módulo orientado a la firma digital. Además, su integración directa con la plataforma de

identidad cubana y con los servicios de firma electrónica existentes no está disponible sin desarrollo adicional.

1.6.2 Análisis comparativo e identificación de brechas

Con el propósito de evaluar el grado de correspondencia entre las soluciones analizadas y los requisitos técnicos, normativos y funcionales definidos para el módulo de seguridad a desarrollar, se realizó un análisis comparativo basado en un conjunto de criterios derivados de las necesidades identificadas con anterioridad. La Tabla 1 resume los criterios considerados en dicho análisis.

Tabla 1. Comparación de soluciones de gestión de claves criptográficas

Criterio	<i>AWS KMS</i>	<i>Azure Key Vault</i>	<i>Google Cloud KMS</i>	<i>HashiCorp Vault</i>
Código abierto	No	No	No	No (<i>BSL</i>)
Despliegue en infraestructura propia	No	No	No	Sí
Soberanía de datos	No	No	No	Sí
<i>API REST</i> con <i>OAuth 2.0/OIDC</i>	Sí	Sí	Sí	Sí (con configuración)
Ciclo de vida completo de claves	Sí	Sí	Sí	Sí
Auditoría integrada	Sí	Sí	Sí	Sí
Complejidad de despliegue	Alta	Alta	Alta	Alta
Costo	Comercial	Comercial	Comercial	Gratuito

Las soluciones comerciales en la nube evidencian la efectividad del patrón de *API REST* con *OAuth 2.0*, *RBAC* y mecanismos de auditoría para la gestión de claves criptográficas [44]. Sin embargo, su aplicación en el contexto cubano resulta limitada debido a la dependencia de infraestructura propietaria externa y a las restricciones asociadas a la soberanía de los datos.

Por su parte, *HashiCorp Vault* elimina esa dependencia al poder desplegarse en infraestructura propia, aunque su complejidad operativa y la ausencia de integración directa con los sistemas de identidad y firma digital limitan su adopción [39, 40].

Del análisis comparativo se identifica la ausencia de una solución que integre la gestión del ciclo de vida de claves, la firma digital, el control de acceso con autenticación federada, la auditoría inmutable y el despliegue simplificado en infraestructura nacional mediante herramientas estándar. Esta brecha permite definir los criterios que debe satisfacer el módulo a desarrollar: código abierto, despliegue en infraestructura propia, integración nativa con proveedores de identidad mediante estándares abiertos, semántica de gestión de claves fundamentada en *KMIP* [38] y complejidad operativa compatible con el ecosistema tecnológico cubano.

1.6.3 Selección de tecnologías para la solución propuesta

La brecha identificada evidencia las carencias, limitaciones y necesidades no resueltas en las soluciones existentes, lo que permite justificar la pertinencia y necesidad de la propuesta. En consecuencia, la selección tecnológica no responde a decisiones arbitrarias, sino a los requisitos que dichas soluciones no satisfacen.

Marco de desarrollo (*framework*)

Para la implementación de la *API REST* del módulo se realizó un análisis comparativo de los marcos de desarrollo más utilizados en la construcción de servicios de este tipo. Se evaluaron *Django*, basado en *Python*; *Express.js*, basado en *Node.js*; *ASP.NET Core*, basado en *.NET*; y *Spring Boot*, basado en *Java*. *Frameworks Java* modernos como *Quarkus* y *Micronaut* no fueron incluidos por presentar menor madurez en el soporte del rol de servidor de recursos *OAuth 2.0* con validación de *JWT* mediante *JWKS* (del inglés, *JSON Web Key Set*), funcionalidad crítica para este sistema. *Spring Security* ofrece esta integración de forma nativa y consolidada, con mayor trayectoria en entornos de producción.

En el análisis se consideraron específicamente los requisitos del módulo, entre ellos: soporte criptográfico sin dependencias externas e integración nativa con mecanismos de autenticación basados en estándares abiertos. Como resultado, se seleccionó *Spring Boot*, un marco de desarrollo que simplifica la creación de

servicios *REST* mediante configuración automática y un servidor *web* embebido [46]. Esta decisión se fundamenta en dos criterios principales.

El primero es la compatibilidad criptográfica nativa. *Java* incorpora la Arquitectura de Criptografía de *Java* (*JCA*, del inglés *Java Cryptography Architecture*), que proporciona implementaciones de los algoritmos requeridos dentro de la propia plataforma, sin necesidad de bibliotecas criptográficas externas adicionales. Esto reduce la superficie de ataque y facilita la auditoría del código [47]. En contraste, los marcos evaluados basados en *Python* y *Node.js* requieren bibliotecas externas para operaciones equivalentes, lo que introduce dependencias adicionales y amplía la superficie de ataque.

El segundo es la integración nativa con *OAuth 2.0/OIDC* mediante *Spring Security*, lo que permite autenticar y autorizar cada solicitud a partir de la validación de *tokens JWT* emitidos por el proveedor de identidad, sin estado adicional [23, 48-50]. Esta integración resulta determinante, dado que el módulo delega completamente la gestión de identidades en un proveedor externo.

Base de datos

Para la persistencia de claves cifradas y registros de auditoría se evaluaron distintas alternativas: *MySQL 8*, *MongoDB 7*, *SQLite* y *PostgreSQL 16*. En el análisis se consideraron los requisitos del modelo de datos del sistema, la garantía de consistencia bajo acceso concurrente y las condiciones de licenciamiento.

A partir de esta evaluación se seleccionó *PostgreSQL 16*, un sistema de gestión de bases de datos relacional de código abierto con soporte completo del modelo *ACID* (del inglés, *Atomicity, Consistency, Isolation, Durability*) [51]. Esta decisión responde a tres criterios.

El primero es el soporte de tipos de datos nativos requeridos por el sistema, entre ellos identificadores *UUID*, marcas de tiempo con zona horaria y texto de longitud variable para el almacenamiento del material cifrado en formato *Base64*.

El segundo criterio es la integridad referencial y la concurrencia. El modelo de datos del sistema es de naturaleza relacional: las entidades que lo componen mantienen relaciones entre sí cuya consistencia debe garantizarse a nivel de motor de base de datos. *PostgreSQL* ofrece soporte completo para claves foráneas y restricciones de integridad, lo que impide que las relaciones entre entidades queden en estado inconsistente ante cualquier operación. Además,

gestiona el acceso concurrente mediante control de concurrencia multiversión (*MVCC*, del inglés *Multiversion Concurrency Control*), que permite múltiples operaciones simultáneas de lectura y escritura sin bloqueos agresivos. *SQLite*, en contraste, bloquea el archivo completo durante operaciones de escritura, lo que lo hace inadecuado para un servicio con múltiples usuarios concurrentes [51].

El tercer criterio es la licencia sin restricciones comerciales. *PostgreSQL* se distribuye bajo la *PostgreSQL License*, una licencia permisiva que no impone condiciones sobre su uso en productos comerciales ni sobre la distribución del software que lo integra. *MySQL*, por su parte, se distribuye bajo licencia *GPL* (del inglés *General Public License*) con excepciones comerciales que pueden generar conflictos en determinados contextos de distribución, mientras que *MongoDB* utiliza la *SSPL* (del inglés *Server Side Public License*), considerada más restrictiva para integraciones en servicios propietarios [51].

Proveedor de identidad

Keycloak 26 es una plataforma de gestión de identidades y acceso de código abierto que implementa de forma completa los protocolos *OAuth 2.0* y *OpenID Connect* [23]. Su selección responde directamente a la brecha identificada en las soluciones analizadas: la ausencia de integración nativa con un proveedor de identidad desplegable en infraestructura propia.

Keycloak cubre esta brecha mediante cuatro características concretas. En primer lugar, la gestión de roles a nivel de cliente permite definir roles específicos por aplicación que se incluyen automáticamente en las *claims* del *JWT* [23]. En segundo lugar, el soporte de autenticación multifactor mediante *TOTP* (del inglés *Time-Based One-Time Password*) añade una capa de seguridad adicional sin requerir desarrollo propio. En tercer lugar, la validación de *tokens* mediante *JWKS* evita consultas constantes al proveedor de identidad durante la autenticación, no introduce latencia en cada operación [48]. Por último, el despliegue en contenedor bajo control total de la organización elimina la dependencia de servicios gestionados en la nube, como *Auth0* u *Okta* [52].

Estrategia de despliegue

Para el despliegue del módulo se seleccionó *Docker* junto con *Docker Compose* como estrategia de orquestación de contenedores. *Docker Compose* permite coordinar todos los servicios que componen el sistema en un entorno

reproducible a partir de un único archivo de configuración [52]. Esta selección responde a dos necesidades concretas derivadas del contexto de despliegue.

La primera es la portabilidad independiente del sistema operativo anfitrión, que permite desplegar el módulo en cualquier servidor *Linux* con *Docker* instalado, sin requerir configuración específica del entorno. La segunda es el aislamiento entre servicios mediante una red interna privada, lo que garantiza que la base de datos no exponga su puerto al exterior y que el proveedor de identidad publique únicamente el puerto necesario para la autenticación. Esto reduce la superficie de ataque a los puertos estrictamente necesarios [52].

1.7 Conclusiones parciales

Como resultado del análisis de los aspectos abordados con anterioridad, se presentan las siguientes conclusiones:

1. La seguridad informática se considera un componente esencial en la protección de la información digital, al garantizar la confidencialidad, integridad y disponibilidad de los datos mediante mecanismos criptográficos y modelos de control de acceso adecuados.
2. La firma digital representa el mecanismo criptográfico que proporciona mayores garantías de autenticidad, integridad y no repudio dentro de los esquemas de firma electrónica, sustentado en el uso de criptografía asimétrica y claves privadas bajo control exclusivo del firmante.
3. Los módulos criptográficos, tanto de hardware como basados en software, permiten proteger el material criptográfico y ejecutar operaciones de cifrado y firma digital dentro de entornos controlados, lo que fortalece la seguridad en la gestión de claves privadas.
4. La gestión del ciclo de vida de las claves criptográficas es un proceso fundamental para preservar la seguridad de los sistemas criptográficos, ya que comprende las operaciones de generación, activación, rotación, revocación y destrucción segura de las claves.
5. Los estándares internacionales y las normativas cubanas establecen los requisitos técnicos, funcionales y de seguridad que deben cumplir los sistemas criptográficos, con el fin de garantizar su confiabilidad, interoperabilidad y validez jurídica en los procesos de firma digital.
6. El análisis de las soluciones existentes evidencia limitaciones asociadas con la soberanía de los datos, la complejidad operativa y la

dependencia de infraestructuras externas, lo cual demuestra que no resuelven de manera integral la problemática identificada en Cuba y justifica la selección de tecnologías orientadas al desarrollo de una solución adaptable a las necesidades nacionales.

CAPÍTULO II. DISEÑO DEL MÓDULO DE SEGURIDAD BASADO EN SOFTWARE

Como continuidad del análisis conceptual, normativo y tecnológico desarrollado previamente, se presenta el diseño del módulo de seguridad por software orientado a la gestión de claves criptográficas y firma digital. Sobre esta base, se establecen los requisitos funcionales y no funcionales del sistema, la arquitectura multicapa y los patrones de diseño que sustentan la solución propuesta. De igual forma, se describen el diseño de la *API REST*, el modelo de base de datos y los mecanismos de autenticación y control de acceso basados en *OAuth 2.0* y *JSON Web Tokens (JWT)*, para garantizar la seguridad, la interoperabilidad y la mantenibilidad del módulo desarrollado.

2.1 Requisitos del módulo de seguridad basado en software

La definición de los requisitos se entiende como una etapa fundamental en el diseño de un sistema *HSM* basado en software, ya que permite establecer las capacidades y condiciones técnicas que debe satisfacer la solución propuesta. En correspondencia con ello, se describen las operaciones y servicios que el sistema debe proporcionar, así como las propiedades de seguridad, rendimiento, interoperabilidad y mantenibilidad necesarias para garantizar un funcionamiento confiable y seguro del módulo desarrollado.

2.1.1 Requisitos funcionales

Los requisitos funcionales describen las capacidades y servicios que el sistema debe proporcionar, así como el comportamiento esperado ante determinadas condiciones de operación [53]. Para el *HSM* basado en software, se establecen los siguientes nueve requisitos funcionales:

- RF 1.** El sistema debe proveer a un usuario la capacidad de generar claves criptográficas y almacenarlas de forma segura.
- RF 2.** El sistema debe proveer a un administrador la capacidad de rotar una clave criptográfica, generar una nueva versión y mantener los metadatos asociadas a la original.
- RF 3.** El sistema debe proveer a un administrador la capacidad de destruir una clave criptográfica, mediante la eliminación permanente e irre recuperable del material de clave privada almacenado.

- RF 4.** El sistema debe proveer al propietario de una clave la capacidad de ejecutar operaciones de firma digital, y garantizar que la clave privada no abandone el módulo en ningún momento durante la operación.
- RF 5.** El sistema debe proveer a un usuario la capacidad de verificar firmas digitales, comparar la firma recibida con la clave pública asociada al identificador de la clave almacenada.
- RF 6.** El sistema debe proveer a un usuario la capacidad de exportar la clave pública en formato estándar para su uso por parte de verificadores externos.
- RF 7.** El sistema debe proveer a un usuario la capacidad de consultar los metadatos y el estado de una clave criptográfica, que incluye algoritmo, tamaño, fecha de creación, estado y propietario.
- RF 8.** El sistema debe proveer a un usuario la capacidad de listar las claves criptográficas almacenadas, con visibilidad diferenciada según su rol.
- RF 9.** El sistema debe proveer a un auditor la capacidad de consultar los registros de auditoría mediante filtros que faciliten la revisión de las operaciones ejecutadas sobre el sistema.

En conformidad con el principio de generación interna de claves establecido en FIPS 140-3, el sistema no permite la importación de claves privadas generadas externamente. Toda clave privada se genera dentro del módulo para garantizar que el material criptográfico nunca haya estado fuera de un entorno controlado. En la Figura 2 se presenta el diagrama de casos de uso del sistema, el cual contempla tres actores diferenciados que reflejan el modelo de control de acceso basado en roles aplicado en el diseño.

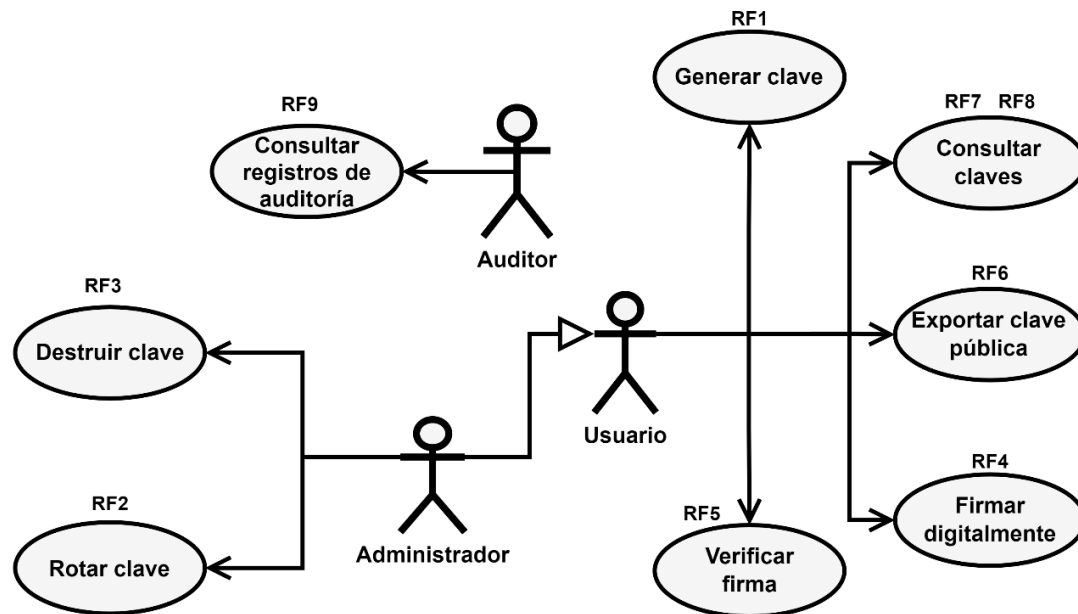


Figura 2. Diagrama de casos de uso del sistema

El “administrador” dispone de acceso completo al ciclo de vida de las claves: puede crear claves, listar todas las del sistema con visibilidad global, consultar sus metadatos, ejecutar operaciones de firma y verificación, rotar claves, y destruirlas. El “usuario” es el actor operativo estándar: puede crear claves propias, listarlas y consultar sus metadatos, firmar datos con sus propias claves, y verificar firmas; sin embargo, no puede rotar ni destruir claves, ni acceder a los registros de auditoría. El “auditor” posee acceso exclusivo de solo lectura a los registros de auditoría y no puede ejecutar ninguna operación sobre las claves. La diferenciación entre el rol “auditor” y el rol “administrador” responde al principio de separación de funciones (*Separation of Duties*). El administrador, encargado de la gestión de claves, no tiene acceso a los registros de auditoría, mientras que el auditor, responsable de su consulta, no puede realizar operaciones criptográficas. Este diseño evita que una sola entidad tenga control total sobre un proceso crítico del sistema.

2.1.2 Requisitos no funcionales

Los requisitos no funcionales describen las características cualitativas del sistema, es decir, cómo debe comportarse más allá de las funciones específicas que ejecuta. En el sistema propuesto, estos requisitos se organizan en cuatro categorías de acuerdo con los atributos de calidad que abordan: seguridad,

portabilidad e interoperabilidad, fiabilidad y usabilidad; los que se aplican de manera transversal a toda su operación.

Requisitos de seguridad

- La clave privada no debe exponerse en ninguna respuesta de la *API*, bajo ninguna circunstancia ni en ningún estado del ciclo de vida de la clave.
- Toda solicitud a la *API* debe estar autenticada mediante un *JWT* válido emitido por el proveedor de identidad configurado, verificado criptográficamente antes de procesar cualquier operación.
- Todas las operaciones sensibles (creación, firma, rotación, destrucción y consulta de claves) deben quedar registradas en un registro de auditoría inmutable.
- El acceso a las operaciones del sistema debe estar controlado por roles, de modo que cada usuario solo pueda ejecutar las operaciones para las que esté autorizado.
- Las claves privadas deben generarse siempre dentro del módulo, sin admitir importación de material criptográfico externo.

Requisitos de portabilidad e interoperabilidad

- El sistema debe ejecutarse en contenedores *Docker* sobre entornos *Linux*.
- El sistema debe integrarse con soluciones de identidad federada mediante *Keycloak* u otros proveedores de identidad (*IdP*) compatibles con los estándares *OAuth 2.0* y *OIDC*, lo que asegura interoperabilidad con plataformas de autenticación corporativas.
- El sistema debe exponer sus servicios a través de una *API REST* estandarizada para garantizar interoperabilidad con clientes externos.

Requisitos de fiabilidad

- El sistema debe incluir un mecanismo de verificación de estado que permita comprobar la operatividad del servicio y de sus componentes críticos (base de datos y servidor de identidad).

Requisitos de usabilidad

- El sistema debe proveer respuestas claras mediante el uso de códigos de estado *HTTP* adecuados y mensajes descriptivos en todos sus *endpoints*.

- El sistema debe ofrecer mecanismos de consulta de auditoría con filtros que faciliten la revisión de operaciones a los usuarios con rol de auditor.

2.2 Arquitectura del sistema

La arquitectura del sistema determina la estructura organizativa de alto nivel del software, así como la distribución e interacción de sus componentes y la lógica que guía las decisiones de diseño [54, 55]. En el caso de un módulo de seguridad por software, la selección de una arquitectura adecuada resulta esencial para garantizar atributos de calidad como la seguridad, la mantenibilidad y la fiabilidad.

2.2.1 Patrón arquitectónico N-Capas

La arquitectura del *HSM* por software se organiza según el estilo arquitectónico de llamada y retorno, característico de sistemas en los que un componente principal invoca servicios de manera controlada, el componente receptor ejecuta la operación correspondiente y posteriormente devuelve una respuesta. Este estilo favorece la modularidad, la mantenibilidad y la claridad en el flujo de control, atributos esenciales en un sistema encargado de gestionar operaciones criptográficas sensibles.

Dentro de este estilo se aplica el patrón N-Capas, el cual organiza el sistema en grupos de componentes clasificados según su nivel de abstracción y responsabilidad. Este patrón separa las capas de la exposición, la lógica de negocio, la seguridad y el acceso a datos, lo que facilita la mantenibilidad, la reutilización y la realización de pruebas independientes de cada nivel del sistema.

La Figura 3 muestra el diagrama basado en responsabilidades que ilustra la arquitectura en capas del sistema.

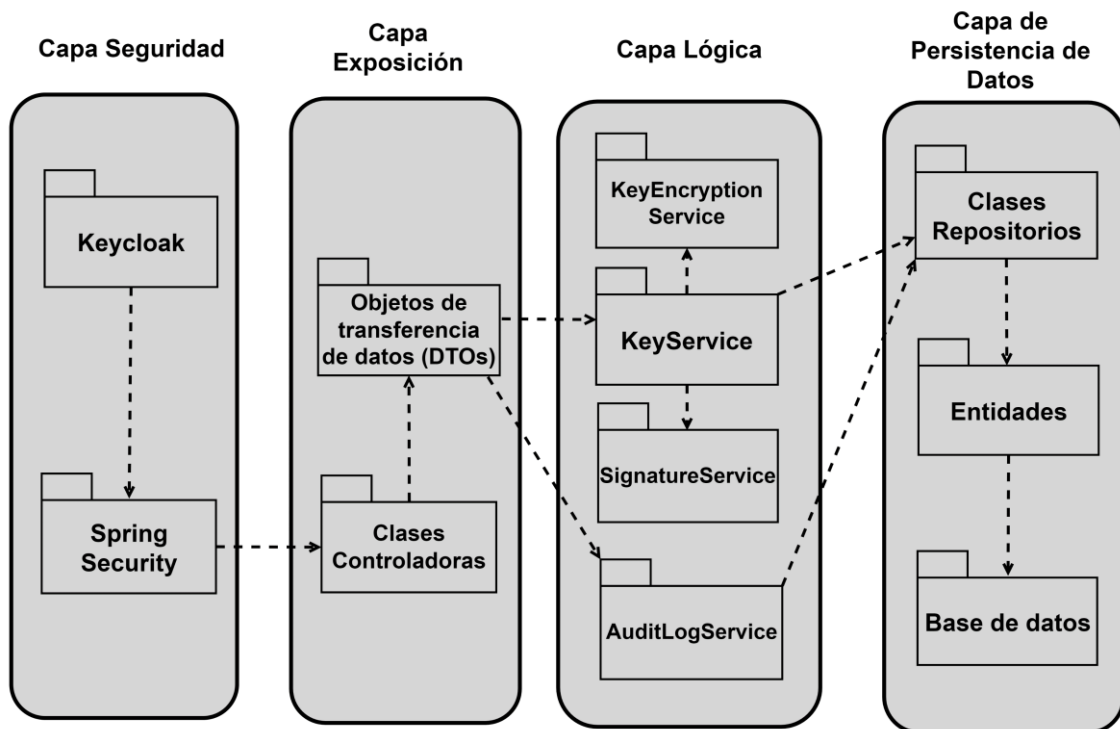


Figura 3. Diagrama N-Capas

En la solución propuesta, el sistema se organiza en cuatro capas con responsabilidades claramente definidas:

Capa de seguridad

- *Keycloak*: proveedor de identidad externo encargado de autenticar usuarios, emitir *tokens* de acceso *JWT*, y aplicar roles y políticas de autorización.
- *Spring Security*: módulo que protege los *endpoints* y gestiona la autorización interna de cada solicitud, mediante tres componentes: *SecurityConfig*, que define las reglas de acceso y establece la política de validación sin estado (*stateless*); *JwtAuthenticationConverter*, que extrae los roles del *JWT* y los registra como autoridades en el contexto de seguridad; y los manejadores *AuditAuthenticationEntryPoint* y *AuditAccessDeniedHandler*, que responden con error de autenticación requerida (401 *Unauthorized*) y error de acceso prohibido (403 *Forbidden*) respectivamente, y registran el intento fallido en el sistema de auditoría.

Capa de exposición

- Clases controladoras (*Controllers*): reciben las peticiones entrantes, validan los datos de entrada y delegan la ejecución en la capa de lógica de negocio.
- Objetos de transferencia de datos (*DTOs*): definen el contrato de la *API* y desacoplan la representación *HTTP* del modelo de dominio interno. Las clases controladoras reciben las solicitudes del cliente y las transfieren a *KeyService* y *AuditLogService*; posteriormente, reciben de estos los *DTOs* de respuesta que son devueltos al cliente.

Capa de lógica de negocio

- Servicios de orquestación: *KeyService*, responsable del ciclo de vida de las claves (creación, rotación, destrucción y control de titularidad de la clave), y *AuditLogService*, encargado de registrar cada operación con su resultado y su huella de integridad.
- Motor criptográfico: conjunto de componentes subordinados a los servicios de orquestación, encargados de ejecutar las operaciones criptográficas mediante *JCA*. *KeyEncryptionService* gestiona el cifrado y descifrado del material de clave privada mediante *AES-256-GCM* (del inglés, *Advanced Encryption Standard* en modo *Galois/Counter Mode*); *SignatureService* ejecuta las operaciones de firma y verificación con *RSA* y *ECDSA* (del inglés, *Elliptic Curve Digital Signature Algorithm*).

Capa de persistencia de datos

- Clases repositorio: *KeyRepository* y *AuditLogRepository* proporcionan la interfaz de acceso estructurado a la base de datos mediante *Spring Data JPA*. *AuditLogRepository* implementa además *JpaSpecificationExecutor*, que permite construir filtros dinámicos sobre los registros de auditoría.
- Entidades: *KeyEntity* y *AuditLogEntity* representan las estructuras de datos persistidas: claves cifradas con sus metadatos y registros de auditoría inmutables, respectivamente.
- Base de datos (*PostgreSQL 16*): almacena las claves cifradas, los registros de auditoría y los metadatos necesarios para el funcionamiento del *HSM*.

La separación entre estas capas responde directamente a los requisitos no funcionales del sistema. La capa de seguridad garantiza que ninguna solicitud sin autenticar acceda a la lógica de negocio. La capa de exposición desacopla el contrato de la *API* de la implementación interna. La capa de lógica de negocio concentra las operaciones criptográficas en un espacio controlado, con una jerarquía interna clara definida entre servicios de orquestación y motor criptográfico. Finalmente, la capa de persistencia abstrae el almacenamiento de material sensible detrás de interfaces tipadas.

2.2.2 Patrones de diseño

El sistema incorpora patrones de diseño que organizan la implementación y refuerzan la separación de responsabilidades establecida por la arquitectura N-Capas. A continuación, se describen los tres patrones identificados.

Patrón *Facade* (Fachada)

El patrón *Facade* tiene como objetivo proporcionar una interfaz simplificada hacia un subsistema complejo, de modo que el cliente no necesite conocer ni interactuar con los componentes internos que ejecutan el trabajo real.

En la Figura 4 se presenta el diagrama de clases del patrón *Facade* aplicado a la capa de exposición del sistema.

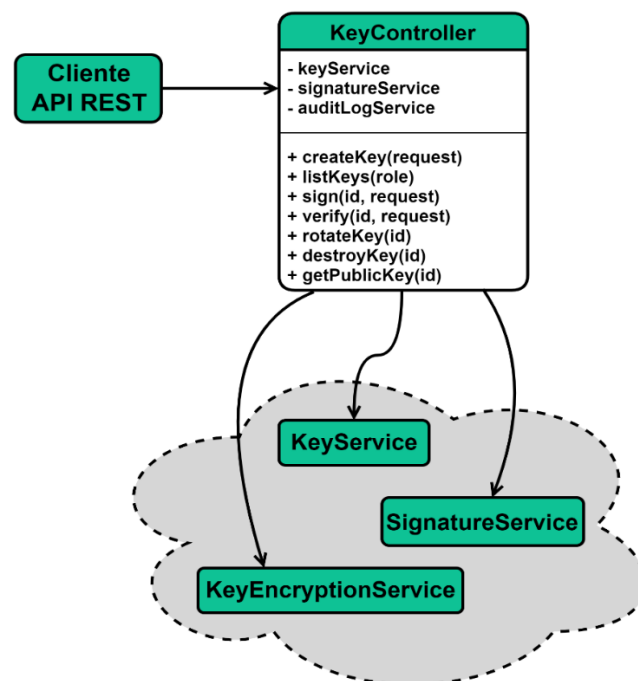


Figura 4. Diagrama de clases del patrón *Facade* (Fachada)

En el *HSM* por software, este patrón se materializa en los controladores de la capa de exposición. *KeyController* expone una interfaz *HTTP* unificada que oculta al cliente *REST* la complejidad del subsistema interno, que incluye la validación de titularidad de la clave, la orquestación del ciclo de vida de las claves, las operaciones criptográficas delegadas al motor y el registro de auditoría de cada operación.

El cliente únicamente interactúa con los *endpoints HTTP* definidos; sin conocer la existencia de *KeyService*, *KeyEncryptionService*, *SignatureService* ni *AuditLogService*. De forma análoga, *AuditController* expone una interfaz de consulta simplificada que oculta la construcción de filtros dinámicos mediante *JpaSpecificationExecutor*.

Patrón Strategy (Estrategia)

El patrón *Strategy* tiene como propósito encapsular una familia de algoritmos intercambiables detrás de una interfaz común, de modo que el algoritmo concreto pueda seleccionarse en tiempo de ejecución sin que el componente *invocante* conozca sus detalles internos.

La Figura 5 muestra el diagrama de clases del patrón *Strategy* aplicado al motor de firma del sistema.

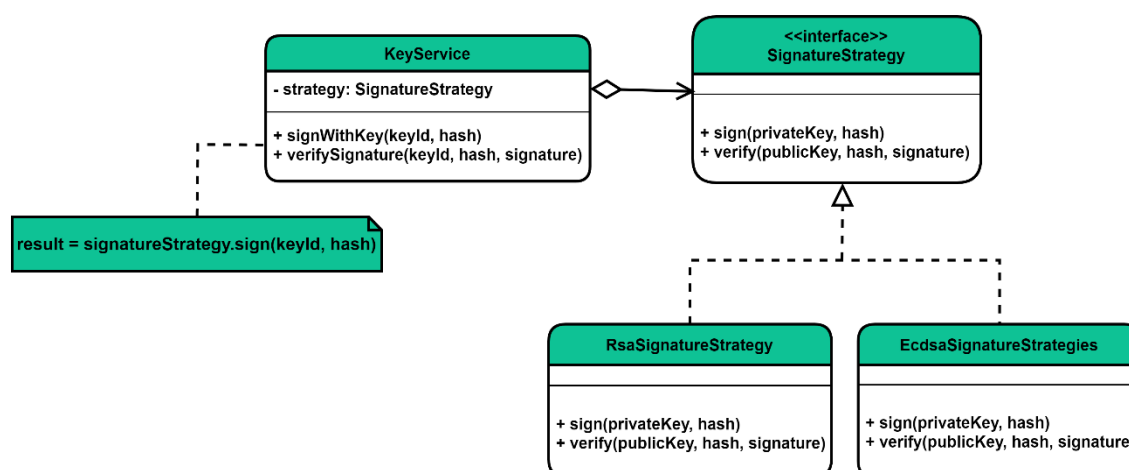


Figura 5. Diagrama de clases del patrón *Strategy* (Estrategia)

En el *HSM* por software, este patrón se manifiesta en el motor criptográfico durante la operación de firma digital. El sistema soporta dos familias de algoritmos con comportamientos internos distintos: *RSA* y *ECDSA*. Ambas estrategias reciben el mismo tipo de entrada y generan igual tipo de salida.

Consecuentemente, *KeyService* delega la operación en *SignatureService* sin conocer qué algoritmo será ejecutado, ya que la selección ocurre en tiempo de ejecución en función del algoritmo asociado a la clave solicitada.

Patrón *DTO* (Objeto de Transferencia de Datos)

El patrón *DTO* (en inglés, *Data Transfer Object*) tiene como finalidad transportar datos entre capas mediante objetos planos sin lógica de negocio, lo que permite desacoplar la representación externa del modelo de dominio interno y evitar la exposición de detalles de implementación al cliente.

En la Figura 6 se presenta el diagrama de clases del patrón *DTO* aplicado al contrato de la *API* del sistema.

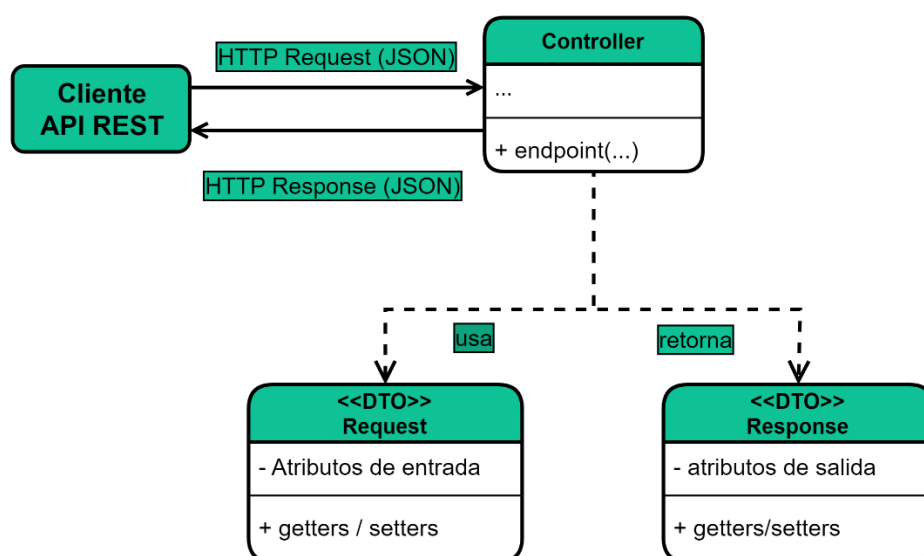


Figura 6. Diagrama de clases del patrón *DTO* (Objeto de Transferencia de Datos)

En el *HSM* por software, este patrón se manifiesta en el contrato de la *API REST*. Las clases *CreateKeyRequest*, *SignRequest* y *VerifyRequest* reciben los datos del cliente *HTTP* y los transfieren a la capa de lógica de negocio sin exponer las entidades internas. Las clases *KeyMetadataResponse*, *SignResponse* y *VerifyResponse* devuelven los resultados al cliente sin revelar los campos internos de *KeyEntity*, en particular el campo *private_key*, que nunca forma parte de ningún *DTO* de respuesta. Este desacoplamiento permite modificar el modelo de dominio interno sin alterar el contrato de la *API*, y viceversa.

2.3 Interfaz de servicios y control de acceso

La *API* adopta la semántica del protocolo *KMIP* para definir las operaciones del ciclo de vida de claves: *Create*, *Get*, *Locate*, *Activate*, *Revoke* y *Destroy*. No se

implementa completamente el protocolo *KMIP* con su capa de transporte binaria; en su lugar, se expone una *API REST* propia que modela dichas operaciones sobre *HTTP/JSON*. Esta decisión permite interoperabilidad con cualquier cliente *HTTP* sin requerir de una biblioteca *KMIP* especializada, al tiempo que mantiene la coherencia semántica con el estándar.

En la Tabla 2 se presentan los 11 *endpoints* del sistema con su método *HTTP*, ruta, descripción funcional y roles autorizados.

Tabla 2. Especificaciones de *endpoints* de la *API REST*

Método	Ruta	Descripción	Rol
POST	/api/keys	Crear clave criptográfica (RSA o ECDSA)	admin, user
GET	/api/keys	Listar claves (admin: todas; user: propias)	admin, user
GET	/api/keys/{id}	Consultar metadatos completos y clave pública	admin, user
DELETE	/api/keys/{id}	Destruir clave (zeroización del material privado)	admin
GET	/api/keys/{id}/public-key	Exportar clave pública en formato PEM	admin, user
POST	/api/keys/{id}/sign	Firmar con clave privada del propietario	admin, user
POST	/api/keys/{id}/verify	Verificar firma con clave pública	admin, user
POST	/api/keys/{id}/rotate	Rotar clave (generar nueva versión)	admin
GET	/api/audit/logs	Consultar registros de auditoría	solo auditor
GET	/api/audit/logs/{id}	Consultar detalle de un registro de auditoría	solo auditor
GET	/api/health	Verificar estado operativo del servicio	Público

Todos los *endpoints* con excepción de */api/health*, requieren un *JWT* emitido por *Keycloak* en el encabezado *Authorization: Bearer <token>*. En ningún caso la clave privada es incluida en las respuestas del sistema.

El *HSM* por software delega la gestión de identidades al servidor *Keycloak*, que actúa como proveedor de identidad central. *Spring Boot* opera como un servidor de recursos (*Resource Server*) en el modelo *OAuth 2.0*, por lo que no gestiona usuarios ni contraseñas propias, sino que valida criptográficamente los *JWT* emitidos por *Keycloak* antes de procesar cualquier solicitud. En la Figura 7 se detalla el diagrama de secuencia del flujo completo de autenticación y autorización de una solicitud a la *API*.

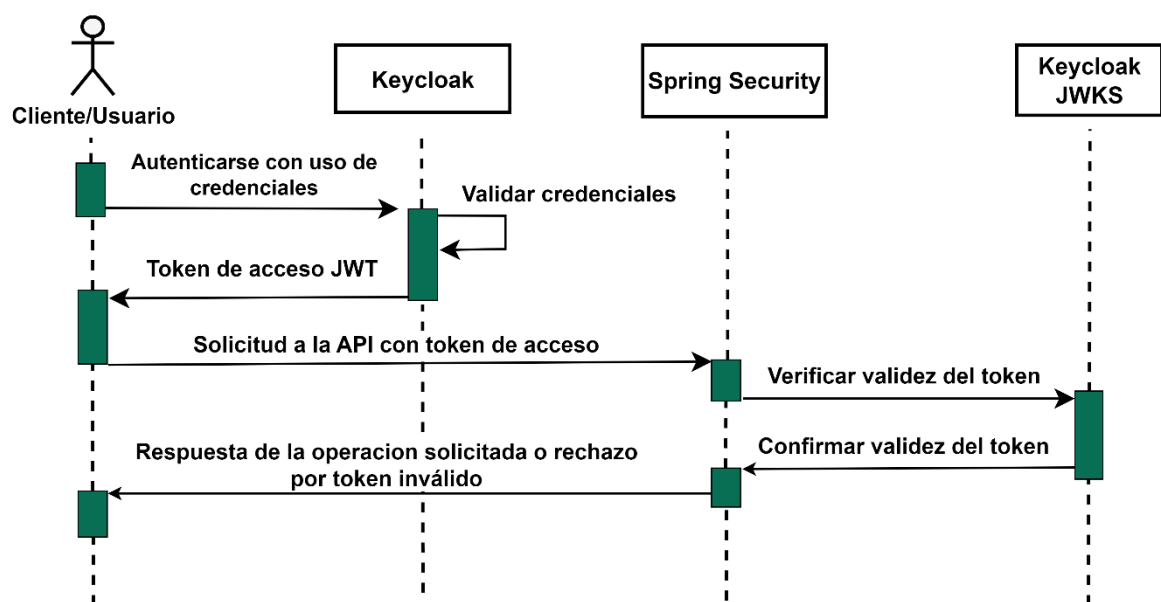


Figura 7. Diagrama de secuencia del flujo de autenticación y autorización

Flujo *OAuth 2.0/OIDC*

Para interactuar con la *API*, el cliente debe obtener previamente un *JWT* de *Keycloak* mediante el flujo *Authorization Code*. *Keycloak* valida las credenciales del usuario, verifica el segundo factor de autenticación si está configurado y emite un *token* de acceso *JWT* firmado con su clave *RSA* privada.

El *JWT* contiene los siguientes *claims* relevantes para el sistema:

- *sub*: identificador único e inmutable del usuario en *Keycloak*. Se utiliza como *owner_id* en la tabla *keys*.
- *preferred_username*: nombre de usuario legible, usado como instantánea en auditoría.

- *realm_access.roles* y *resource_access.<client>.roles*: roles asignados al usuario, que determinan los permisos de acceso a cada *endpoint*.

Validación sin estado del JWT

Spring Security valida la firma del *JWT* en cada solicitud mediante la consulta al *endpoint JWKS*. *JWKS* es el conjunto de claves públicas que *Keycloak* expone para que los servidores de recursos verifiquen criptográficamente los *tokens* que emite y que están disponible en la ruta */realms/realm/protocol/openid-connect/certs*. La validación es sin estado: no existe sesión en el lado del servidor ni base de datos de *tokens* activos. Cuando el *JWT* expira, la solicitud es rechazada con un error de autenticación requerida (*401 Unauthorized*). Por el contrario, si el *JWT* es válido pero el rol es insuficiente para la operación solicitada, la respuesta es un error de acceso prohibido (*403 Forbidden*).

Mecanismo de titularidad de la clave

El campo *owner_id* de la tabla *keys* se asigna en el momento de la creación con el valor del *claim sub* del *JWT* del usuario creador. Dado que el *owner_id* se extrae siempre del *JWT*, firmado por *Keycloak* y no modificable por el cliente, ningún usuario puede declararse propietario de una clave que no haya creado. Al ejecutar la operación de firma, el sistema compara el *sub* del *JWT* del solicitante con el *owner_id* almacenado en la tabla *keys*. Si no coinciden, la operación es rechazada con un error de acceso prohibido (*403 Forbidden*), independientemente del rol del solicitante. Este mecanismo garantiza que ni siquiera el administrador del sistema pueda firmar en nombre de otro usuario sin disponer de sus credenciales de autenticación.

2.4 Modelo de base de datos

El *HSM* por software utiliza *PostgreSQL 16* como motor de base de datos relacional para persistencia del material criptográfico y la trazabilidad de las operaciones.

En la Figura 8 se presenta el diagrama del modelo físico de base de datos con las dos tablas y sus relaciones.

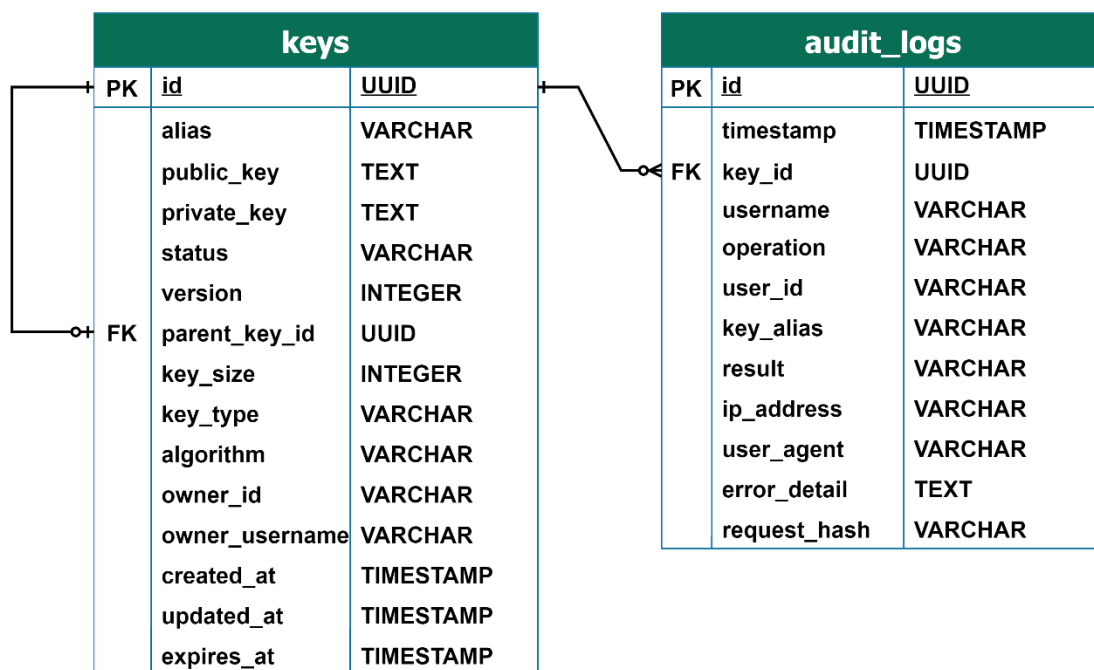


Figura 8. Diagrama del modelo físico de base de datos

El modelo de datos se reduce a dos tablas: *keys*, que concentra todo el material criptográfico que gestiona el sistema, y *audit_logs*, que registra de forma inmutable cada operación ejecutada. Esta reducción responde a una decisión de diseño deliberada: la gestión de identidades, roles y permisos se delega íntegramente a *Keycloak*, por lo que la incorporación de tablas adicionales de usuarios o roles duplicaría información que ya gestiona el proveedor de identidad y generaría una segunda fuente de verdad susceptible de inconsistencia.

Como resultado, en el módulo únicamente persiste el identificador de usuario de *Keycloak* (el campo *sub* del *JWT*), denominado *owner_id* como atributo de las claves, con fines de trazabilidad y control de titularidad de la clave (*ownership*).

Tabla *keys*

Almacena los pares de claves criptográficas generadas por el *HSM*. Cada registro agrupa la clave pública en formato *Base64*, el material de clave privada cifrado con *AES-256-GCM*, los metadatos descriptivos (algoritmo, tamaño, tipo y alias) y los atributos de ciclo de vida (estado, versión, referencia a la versión anterior y fechas de creación, modificación y expiración).

El campo *owner_id* vincula cada clave con el usuario de *Keycloak* que la generó, sin almacenar información de identidad propia del módulo. La columna *private_key* nunca se devuelve al cliente en ninguna respuesta. Cuando una

clave alcanza el estado *DESTROYED*, su contenido se sobrescribe con ceros y el registro se conserva para garantizar la trazabilidad histórica.

Tabla *audit_logs*

Registra todas las operaciones ejecutadas sobre el sistema. La tabla es de tipo *append-only*, por lo que ningún registro puede ser modificado ni eliminado una vez creado. Cada entrada captura la identidad del solicitante, la operación ejecutada, la clave involucrada, el resultado obtenido y el contexto técnico de la solicitud, que incluye la dirección *IP* y el agente *HTTP* del cliente.

Para garantizar la integridad del registro, cada entrada incorpora un *hash SHA-256* del *payload* de la solicitud, lo que permite detectar cualquier alteración posterior. Tanto las operaciones exitosas como los intentos fallidos y los rechazos por control de acceso quedan registrados.

2.5 Conclusiones parciales

El diseño del módulo de seguridad por software desarrollado en el presente capítulo define los fundamentos arquitectónicos, las bases estructurales y funcionales del sistema. A partir del análisis realizado, se derivan las siguientes conclusiones:

1. Los requisitos funcionales y no funcionales definidos delimitan de forma precisa el comportamiento del *HSM* por software y garantizan la generación, gestión y uso seguro de claves criptográficas bajo principios estrictos de seguridad, control de acceso y trazabilidad.
2. La adopción de una arquitectura multicapa junto con la aplicación de patrones de diseño permite una clara separación de responsabilidades y un bajo acoplamiento entre componentes, lo que fortalece la mantenibilidad, escalabilidad y robustez del módulo ante futuros cambios.
3. La *API* estandarizada basada en *REST* y alineada con *KMIP* garantiza la interoperabilidad del sistema, integra mecanismos de control de acceso basado en roles y refuerza la seguridad en la exposición de servicios criptográficos.
4. El modelo de datos simplificado, centrado en las entidades de claves y auditoría, asegura un almacenamiento seguro, trazable e inmutable de las operaciones criptográficas.

5. El uso de *OAuth 2.0/OIDC* con *Keycloak* como proveedor de identidad, junto con validación sin estado de *JWT* y control de acceso basado en roles, garantiza un modelo de seguridad sólido y desacoplado.
6. El diseño definido satisface los criterios de seguridad, mantenibilidad e interoperabilidad establecidos, lo que lo convierte en una base técnica suficiente para la implementación efectiva del módulo.

CAPÍTULO III. IMPLEMENTACIÓN DEL MÓDULO DE SEGURIDAD POR SOFTWARE

En esta sección se describen los mecanismos internos de implementación del módulo de seguridad por software. A partir del diseño establecido previamente, donde se definieron los requisitos, la arquitectura por capas, la *API REST* y el modelo de datos, se expone la forma en que dichos elementos se materializan en el código. En correspondencia con ello, se desarrolla la implementación de los mecanismos asociados a la gestión del ciclo de vida de las claves criptográficas, la protección del material criptográfico en reposo, la ejecución y validación de firmas digitales, el registro inmutable de las operaciones realizadas por el sistema y el despliegue de los componentes en un entorno basado en contenedores. Como resultado, se obtiene la implementación efectiva que respalda las decisiones de diseño definidas con anterioridad.

3.1 Implementación del ciclo de vida de claves criptográficas

El ciclo de vida de una clave criptográfica comprende el conjunto de estados y transiciones por los que esta atraviesa desde su generación hasta su destrucción definitiva, así como las operaciones que determinan cada cambio de estado. Una gestión adecuada del ciclo de vida garantiza que las claves solo puedan emplearse durante períodos autorizados y bajo condiciones seguras de utilización.

3.1.1 Generación de claves

Las claves criptográficas se generan exclusivamente dentro del módulo, de modo que el material sensible nunca resida fuera de este durante el proceso de creación. El sistema admite cuatro configuraciones criptográficas: *RSA* de 3072 bits, *RSA* de 4096 bits, *ECDSA* sobre la curva P-256 y *ECDSA* sobre la curva P-384. El sistema no admite claves *RSA* de 2048 bits, en conformidad con la discontinuación de este tamaño a partir del año 2030, por la insuficiente fortaleza de seguridad que proporciona.

3.1.2 Rotación de claves

La rotación consiste en la generación de un nuevo par de claves que conserva el mismo alias, algoritmo y tamaño de la clave original. La nueva clave se registra con el valor *version* = *versión_anterior* + 1 y el campo *parent_key_id* referencia el identificador de clave rotada, lo que permite mantener la trazabilidad completa del linaje criptográfico a lo largo de sus versiones. Una vez creada exitosamente

la nueva versión, la clave anterior pasa inmediatamente al estado *REVOKED*. No obstante, las firmas generadas con versiones previas continúan siendo verificables mediante la correspondiente clave revocada, sin requerir el mantenimiento de ningún estado adicional en el sistema.

3.1.3 Destrucción segura y zeroización

La destrucción de claves se implementa mediante el mecanismo de zeroización. Para ello, el campo *private_key* almacenado en la base de datos se sobrescribe con un arreglo de ceros de igual longitud que la clave privada cifrada original. El registro asociado permanece en el sistema con *status=DESTROYED*, a fin de preservar la trazabilidad de las operaciones dentro del registro de auditoría.

La zeroización también se aplica sobre el material sensible residente en memoria durante cada operación de firma. Inmediatamente después de ejecutarse la operación criptográfica, los bytes correspondientes a la clave privada descifrada se sobrescriben con el valor 0, lo que reduce al mínimo la ventana temporal en la que dicho material permanece accesible en la memoria del proceso.

3.2 Protección de claves privadas en reposo

Una propiedad fundamental del módulo consiste en que las claves privadas nunca se almacenan en texto claro. Toda clave privada se cifra antes de persistirse en la base de datos y únicamente se descifra en memoria durante el intervalo de tiempo estrictamente necesario para ejecutar la operación criptográfica solicitada.

3.2.1 Arquitectura de clave de cifrado de claves

El sistema implementa una arquitectura criptográfica de dos niveles. Las claves privadas de los usuarios se almacenan cifradas en la base de datos mediante una clave de cifrado de claves, que constituye un segundo nivel, encargada de proteger el material criptográfico operativo. Este patrón se fundamenta en el principio esencial para la gestión segura de claves criptográficas que establece la separación entre claves de protección y claves de trabajo.

La *KEK* se carga, durante el arranque del sistema, a partir de una variable de configuración segura del servidor. Una vez cargada, permanece exclusivamente en la memoria del proceso y nunca se escribe en la base de datos, en el sistema de archivos ni en ningún registro de la aplicación. Si dicha variable no se encuentra configurada correctamente al iniciar, el sistema interrumpe el proceso

de arranque, lo que garantiza que el módulo no opere sin material criptográfico de protección válido.

La separación entre la clave de cifrado de claves y las claves de datos implica que un acceso no autorizado a la base de datos no resulta suficiente para comprometer las claves privadas almacenadas, ya que el atacante requeriría además la *KEK*, la cual nunca reside fuera de la memoria del proceso. En los *HSM* certificados, esta clave suele encontrarse protegida mediante hardware especializado, como un módulo de plataforma confiable (*TPM*, del inglés *Trusted Platform Module*), un chip dedicado que protege el material criptográfico dentro del propio hardware del servidor. En la presente implementación basada en software, dicha función se logra mediante el uso de variables de entorno seguras que representan la aproximación más efectiva en ausencia de hardware especializado.

3.2.2 Algoritmo de cifrado

El algoritmo empleado para el cifrado de las claves privadas es *AES-256-GCM*. *AES-256* proporciona el mayor nivel de seguridad dentro del estándar, mientras que el modo *GCM* convierte el esquema en cifrado autenticado (*Authenticated Encryption with Associated Data, AEAD*). Además, al cifrar el texto, se genera un código de autenticación (*AuthTag*) de 16 bytes que permite detectar cualquier modificación de la clave privada. Si algún byte del texto cifrado almacenado en la base de datos es alterado, el proceso de descifrado falla antes de devolver información al sistema, lo que impide el uso de material manipulado.

Esta propiedad diferencia a *AES-256-GCM* de otros modos de operación, como *AES-CBC*, que proporciona confidencialidad, pero no autenticación, o *AES-ECB*, que revela patrones al generar bloques cifrados idénticos para entradas iguales. Por estas razones, ambos modos fueron descartados en favor de *GCM*.

3.2.3 Flujo de cifrado y descifrado

Durante la creación de una clave, el sistema genera el par criptográfico mediante *JCA* y serializa la clave privada en formato *PKCS#8*. Para cifrarla, el esquema *AES-256-GCM* requiere tres elementos además del material a proteger: la *KEK*, un vector de inicialización y datos asociados autenticados.

El vector de inicialización (*IV*) es un valor aleatorio de 12 bytes obtenido mediante *SecureRandom*, un generador que se alimenta de ruido físico del hardware para producir valores verdaderamente impredecibles. Se genera uno nuevo por cada

operación, lo que garantiza que cifrados distintos del mismo material produzcan resultados diferentes, aun cuando la *KEK* sea la misma. El *IV* no es secreto y se almacena junto con el resultado cifrado, ya que es necesario para revertir la operación.

Los datos asociados autenticados (*AAD*, del inglés *Additional Authenticated Data*) toman el valor “*alias:ownerId*”, por ejemplo “*firma-contratos:a1b2c3d4-e5f6-...*”. El *AAD* no se cifra, pero sí se autentica. El descifrado solo tiene éxito si el *AAD* presentado coincide exactamente con el utilizado al cifrar. Esto vincula criptográficamente cada clave privada cifrada con la fila específica de la base de datos a la que pertenece. En consecuencia, si el contenido cifrado de un usuario A se copiara a la fila correspondiente a un usuario B, el descifrado fallaría debido a la discrepancia en el identificador del propietario. De este modo, se previenen ataques de reordenamiento de filas sin necesidad de incorporar código adicional de validación.

Desde el punto de vista de su almacenamiento y posterior recuperación, el resultado del proceso de cifrado se representa como un bloque de datos binarios, denominado *blob* cifrado (*blob*, del inglés *Binary Large Object*), codificado en *Base64* con el siguiente formato:

Base64(IV [12 bytes] || CipherText [N bytes] || AuthTag [16 bytes])

El *AuthTag* de 16 bytes, generado por el modo *GCM*, permite detectar cualquier alteración del bloque almacenado: si un byte es modificado, el descifrado falla antes de devolver información al sistema. Este bloque cifrado se almacena en la base de datos y los bytes del material sensible en memoria se sobrescriben con ceros de forma inmediata.

Durante la operación de firma, el sistema recupera el bloque cifrado de la base de datos, decodifica el contenido *Base64*, extrae el *IV* de los primeros 12 bytes y realiza el descifrado mediante la verificación del *AuthTag* y el *AAD*. Posteriormente, reconstruye el objeto de clave a partir del formato *PKCS#8*, ejecuta la operación criptográfica y sobrescribe los bytes con ceros. De este modo, el material sensible permanece en memoria únicamente durante el tiempo estrictamente necesario para la operación.

En la Figura 9 se presenta el diagrama que detalla el proceso de cifrado

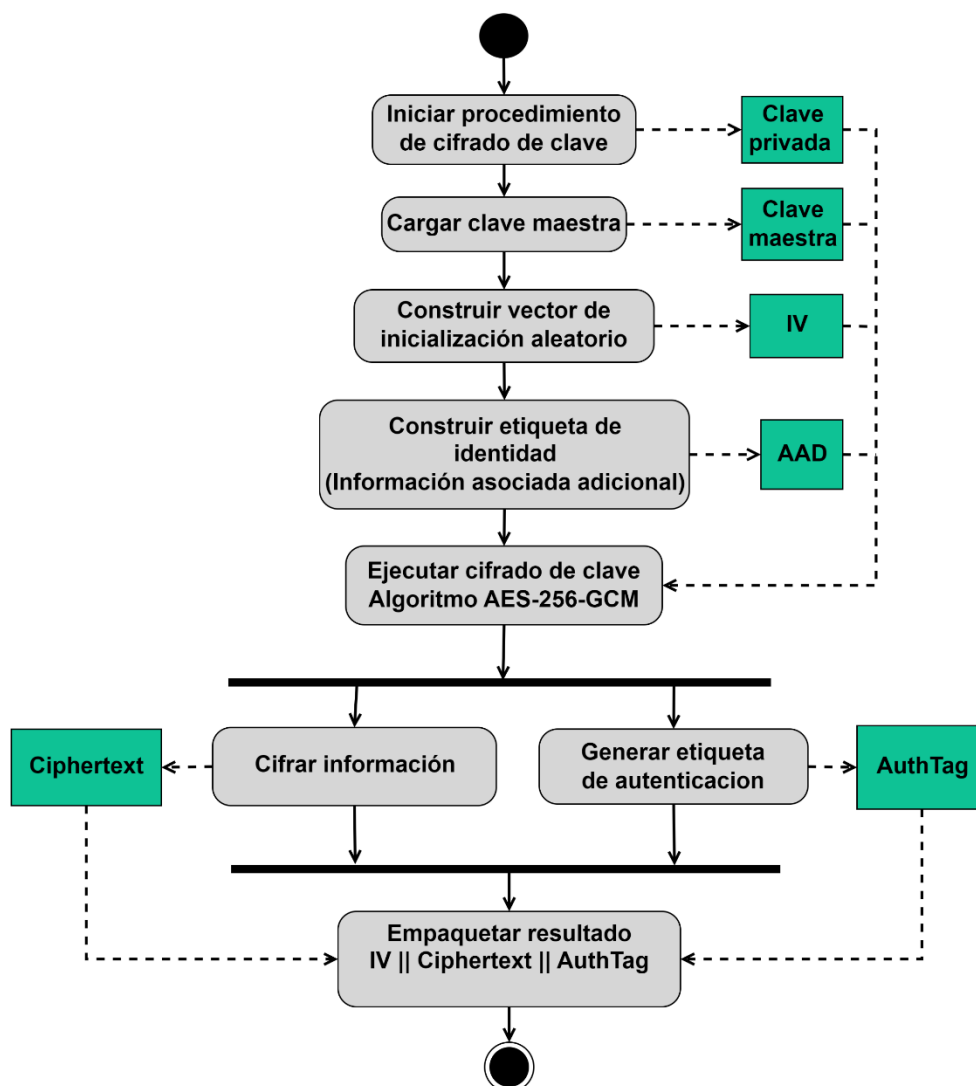


Figura 9. Diagrama del proceso de cifrado

3.3 Firma digital y verificación

La firma digital se reconoce como la operación fundamental del módulo, al garantizar la autenticidad, la integridad y el no repudio de los datos procesados. Su implementación comprende los algoritmos criptográficos empleados según el tipo de clave, el modelo de interacción entre cliente y servidor para la generación de firmas y las normas que rigen la verificación de firmas existentes.

3.3.1 Algoritmos criptográficos de firma y modelo de operación

Algoritmo RSA

RSA constituye uno de los algoritmos de firma asimétrica más utilizados en la industria. Su seguridad se fundamenta en la dificultad de factorizar números enteros de gran tamaño. El sistema implementado admite claves RSA de 3072 y 4096 bits. Para producir una firma, RSA no opera directamente sobre el

documento, sino sobre el resumen criptográfico, que se obtiene al aplicar una función *hash* como *SHA-256*, *SHA-384* o *SHA-512* al contenido. Esto reduce el volumen de datos a procesar, disminuye la complejidad del procesamiento y garantiza que la firma sea sensible ante cualquier alteración del documento original.

Dado que el módulo recibe el resumen previamente calculado por el cliente, la implementación de *RSA* incorpora un paso adicional antes de la firma: el empaquetado del resumen junto con un identificador del algoritmo *hash* utilizado, conforme al estándar *PKCS#1*. Esto da lugar a la formación de un bloque estandarizado denominado *DigestInfo*, que puede ser interpretado por cualquier verificador. Este empaquetado garantiza la interoperabilidad de las firmas generadas y su verificabilidad por cualquier implementación *RSA* conforme al estándar, independientemente del lenguaje o la plataforma, sin requerimientos especiales de configuración.

Algoritmo ECDSA

ECDSA es el algoritmo de firma basado en criptografía de curvas elípticas. Su seguridad se sustenta en la dificultad del problema del logaritmo discreto sobre curvas elípticas, lo que le permite ofrecer un nivel de seguridad equivalente al de *RSA* con claves considerablemente más cortas.

El sistema admite dos curvas: *P-256*, que proporciona un nivel de seguridad equivalente al de *RSA* de 3072 bits, y *P-384*, equivalente a *RSA* de 7680 bits. A diferencia de *RSA*, *ECDSA* incorpora de forma nativa el resumen del documento en su operación matemática, por lo que no requiere el paso adicional de empaquetado descrito previamente para *RSA*. La firma resultante consiste en un par de valores numéricos que el receptor utiliza junto con la clave pública correspondiente para verificar su autenticidad.

Modelo de firma desacoplada

El módulo implementa el modelo de firma desacoplada, debido a que este constituye el esquema propio de los componentes criptográficos especializados. En este modelo, el cliente aporta únicamente el resumen del documento y el módulo devuelve la firma digital correspondiente sin procesar el documento original.

En la Figura 10, se presenta el flujo del proceso de firma digital implementado bajo el modelo de firma desacoplada.

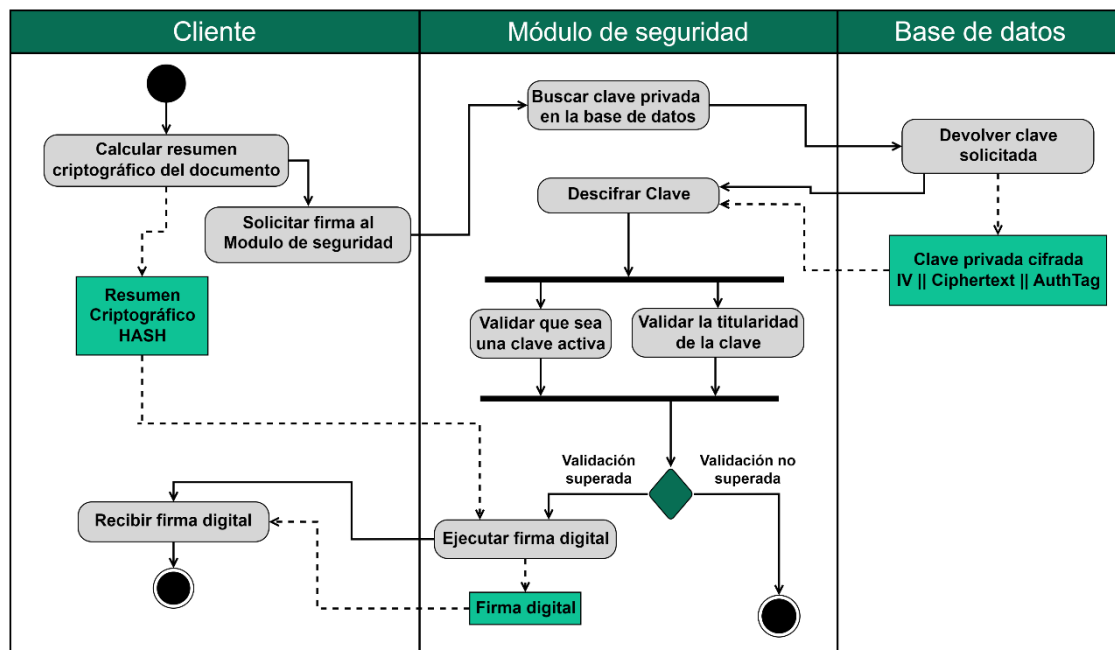


Figura 10. Diagrama del proceso de firma digital

Este enfoque permite firmar documentos de cualquier tamaño sin necesidad de transmitirlos al servidor, reduce la superficie de ataque y es coherente con el comportamiento de los *HSM* certificados. En escenarios donde una aplicación requiere una firma embebida en un formato de documento como PDF, la arquitectura adecuada consiste en que dicha aplicación extraiga el resumen, solicite la firma al *HSM* e inserte posteriormente el resultado en el archivo correspondiente.

3.3.2 Verificación de firmas

La verificación de firmas emplea únicamente la clave pública asociada a la clave utilizada para la firma, sin acceder en ningún momento al material privado almacenado de forma cifrada en la base de datos. El proceso reconstruye internamente el mismo bloque utilizado durante la operación de firma y comprueba si la firma recibida corresponde con dicho bloque y con la clave pública proporcionada. Este procedimiento permite garantizar la autenticidad del origen y la integridad del documento.

Todo usuario autenticado puede verificar una firma realizada con cualquier clave del sistema sin necesidad de ser su propietario. Las claves en estado revocado o expirado pueden participar en la verificación, ya que pueden existir firmas válidas generadas antes de que la clave cambiara de estado. Las claves destruidas, en cambio, no participan en la verificación, debido a que su material

público también ha sido eliminado del sistema. Cuando la firma no corresponde al resumen presentado, el sistema devuelve una respuesta en la que se indica que la verificación no fue satisfactoria. Esto no constituye un error del sistema, sino el resultado válido de una comprobación negativa.

3.4 Registro de auditoría

El sistema implementa un registro inmutable de todas las operaciones ejecutadas sobre el módulo, lo que garantiza el no repudio y la trazabilidad de cada acción sobre las claves criptográficas. El acceso a estos registros se encuentra restringido exclusivamente al rol de auditor, el cual no dispone de acceso a ninguna otra función del *HSM*, de modo que ningún rol pueda ejecutar una operación y, posteriormente, alterar su registro.

3.4.1 Registro de solo inserción

La tabla de registros de auditoría opera bajo el principio de solo inserción, conocido como *append-only* en la literatura de seguridad, lo que implica que ninguna operación del sistema puede modificar ni eliminar registros existentes. Cada entrada incluye los siguientes atributos: un identificador único, una marca de tiempo, el identificador del usuario obtenido del *token* de autenticación, el nombre de usuario, el tipo de operación ejecutada, el identificador y alias de la clave sobre la que se realizó la operación, el resultado de la operación, la dirección de red del cliente y, en caso de fallo, el detalle del error.

Este modelo garantiza la integridad estructural del historial de operaciones y permite la reconstrucción completa de la actividad del sistema sin ambigüedades ni pérdidas de información.

3.4.2 Operaciones auditadas

El sistema registra de forma integral todas las operaciones ejecutadas sobre el *HSM*, tanto las que concluyen con éxito como las rechazadas por los controles de autorización. Cuando una solicitud es rechazada por insuficiencia de rol o por violación del principio de titularidad de la clave, el registro documenta el motivo del rechazo. En los sistemas de gestión criptográfica, este principio establece que solo el propietario de una clave puede ejecutar operaciones sobre ella. Esto permite reconstruir la secuencia completa de operaciones sobre cualquier clave, así como los intentos no autorizados, sin posibilidad de que dichos registros hayan sido eliminados o alterados.

Los filtros disponibles para la consulta paginada de registros incluyen: identificador de usuario, nombre de usuario, identificador de clave, tipo de operación, resultado y rango de fechas.

3.5 Modelo de despliegue

El modelo de despliegue describe la organización estructural de los componentes del sistema en nodos de ejecución, así como los canales de comunicación establecidos entre ellos. En un módulo de seguridad orientado a la firma digital en entornos distribuidos, esta definición resulta crítica, ya que determina los límites de confianza, la custodia de las claves y los flujos de autenticación y autorización.

La Figura 11 muestra el diagrama de despliegue de la solución propuesta y las relaciones de comunicación entre sus componentes.

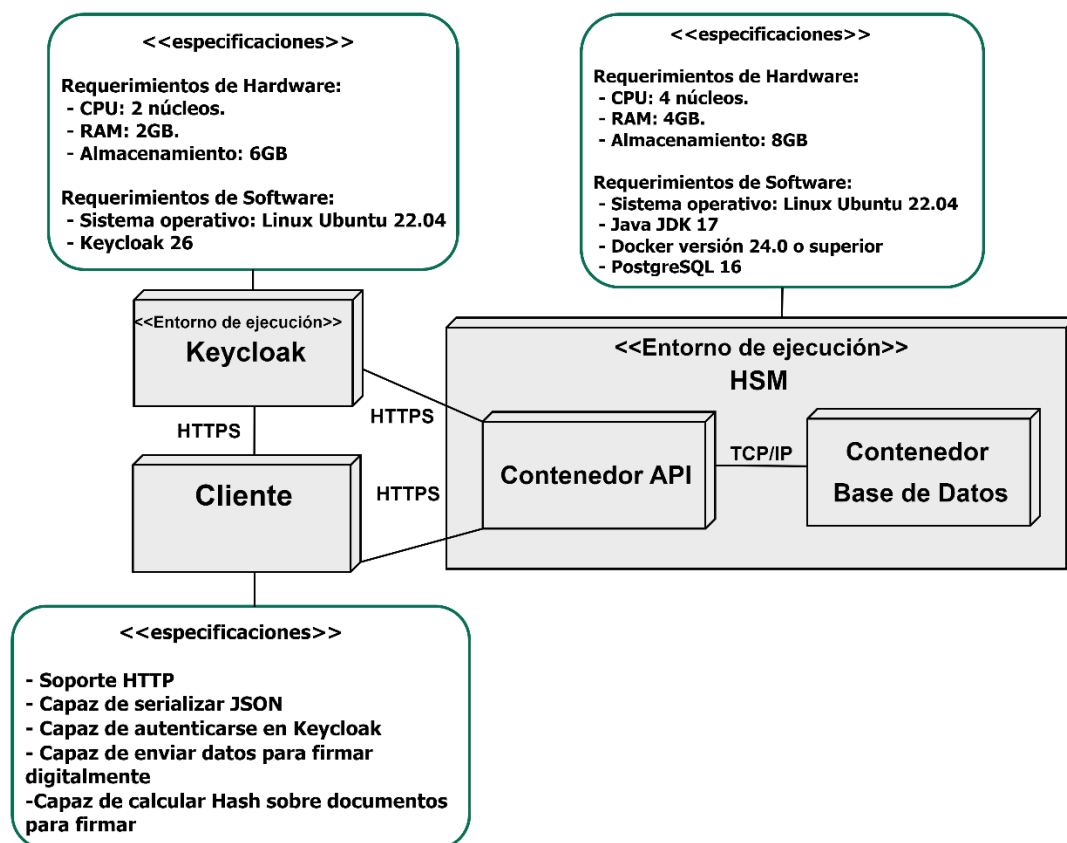


Figura 11. Diagrama del modelo de despliegue

El sistema se distribuye entre dos servidores dedicados: el *Appliance HSM*, denominación que designa a un servidor configurado y dedicado exclusivamente a una función específica, en este caso alojar el módulo de seguridad, y el servidor *Keycloak*, que actúa como proveedor de identidad. Esta separación física garantiza que la clave maestra de cifrado residente en memoria del *Appliance*

HSM no coexista con ningún otro servicio, y que un compromiso del servidor de identidad no implique acceso directo al material criptográfico. Ambos servidores ejecutan *Linux Ubuntu 22.04 LTS* como sistema operativo.

Componentes del sistema

Appliance HSM. Servidor dedicado exclusivamente al módulo de seguridad. Está compuesto por dos contenedores *Docker* que se comunican a través de una red interna:

- Contenedor de aplicación: ejecuta *Spring Boot*, contiene la lógica criptográfica y expone la *API REST*.
- Contenedor de base de datos: ejecuta *PostgreSQL 16* y almacena las claves cifradas y los registros de auditoría.

La *KEK* reside exclusivamente en la memoria del proceso *Spring Boot* y se carga desde una variable de entorno durante el arranque. La base de datos nunca tiene acceso a *KEK*, dado que únicamente almacena material previamente cifrado. El *Appliance* requiere una configuración mínima de 2 núcleos de CPU, 4 GB de memoria RAM y 8 GB de almacenamiento. El entorno de ejecución incluye *Java JDK 17* y *Docker 24.0* o superior.

Servidor Keycloak. Servidor dedicado al proveedor de identidad, separado físicamente del *Appliance HSM*. Gestiona usuarios, roles y la emisión de *tokens JWT* mediante *OIDC*. Expone el *endpoint JWKS* el cual es consultado por el *Appliance HSM* para la validación criptográfica de cada *JWT* recibido. El servidor requiere una configuración mínima de 2 núcleos de CPU, 2 GB de memoria RAM y 6 GB de almacenamiento.

Comunicación entre componentes

El cliente se comunica con el servidor *Keycloak* a través de *HTTPS* para obtener un *token JWT*, e incluye dicho *token* en cada solicitud a la *API* del *Appliance HSM* también mediante *HTTPS*. El *Appliance HSM* consulta el *endpoint JWKS* del servidor *Keycloak* a través de *HTTPS* para validar criptográficamente cada *JWT* recibido. La comunicación entre el contenedor de aplicación y el contenedor de base de datos se realiza mediante *JDBC (Java Database Connectivity)*, la interfaz estándar de *Java* para ejecutar consultas y operaciones sobre bases de datos relacionales. Esta comunicación utiliza *TCP/IP (Transmission Control Protocol/Internet Protocol)*, el protocolo de red que garantiza la entrega ordenada

y confiable de datos entre dos extremos, a través de la red interna de *Docker*, sin exposición al exterior.

Requisitos del lado del cliente

Para integrarse con el sistema, el cliente debe soportar *HTTP/1.1* o superior con *TLS 1.2* como mínimo, ser capaz de serializar y deserializar *JSON*, autenticarse en *Keycloak* mediante el flujo *Authorization Code* e incluir el *JWT* resultante en cada petición.

Derivado del modelo de firma desacoplada, el cliente debe calcular localmente el resumen criptográfico del documento mediante *SHA-256*, *SHA-384* o *SHA-512* antes de invocar la operación de firma. El módulo recibe únicamente dicho resumen y devuelve la firma digital correspondiente. El cliente conserva el documento original junto con la firma resultante y los distribuye al receptor según el protocolo de la aplicación.

3.6 Conclusiones parciales

A continuación, se presentan las conclusiones derivadas de la implementación del módulo de seguridad por software, las que sintetizan los resultados obtenidos en cada área funcional del sistema y destacan los aspectos más relevantes.

- 1 El ciclo de vida de claves criptográficas implementa un conjunto de estados y transiciones controladas que garantizan la gestión segura del material criptográfico desde su generación hasta su destrucción definitiva. La destrucción incluye zeroización de la clave privada cifrada en la base de datos y del material en memoria.
- 2 El sistema establece un esquema de protección de claves privadas basado en cifrado simétrico autenticado, que asegura la confidencialidad e integridad del material criptográfico durante su almacenamiento.
- 3 El módulo implementa un mecanismo de firma digital y verificación basado en algoritmos asimétricos estándar, que garantiza la autenticidad, integridad y no repudio de las operaciones criptográficas realizadas.
- 4 El sistema de auditoría implementa un modelo *append-only* inmutable que asegura la trazabilidad completa de las operaciones y preserva el principio de no repudio dentro del módulo de seguridad.

- 5 El modelo de despliegue define una arquitectura distribuida con separación de responsabilidades entre componentes, lo que delimita los dominios de confianza y refuerza el aislamiento del material criptográfico y de los servicios de identidad.
- 6 La implementación presentada consolida una arquitectura de seguridad integral basada en principios criptográficos, aislamiento de componentes y control de acceso, que fundamenta el proceso de validación del sistema y la evaluación del cumplimiento de los requisitos de seguridad, consistencia y robustez definidos.

CAPÍTULO IV. VALIDACIÓN DE LA SOLUCIÓN

El presente capítulo presenta la validación de la solución implementada para el módulo de seguridad por software, orientado a la gestión del ciclo de vida de claves criptográficas y a la firma digital. Su objetivo es comprobar el correcto funcionamiento del sistema y demostrar el cumplimiento de los requisitos funcionales y de seguridad establecidos durante su diseño.

Con este propósito se aplicaron tres niveles de prueba complementarios:

- Las pruebas unitarias, dirigidas a verificar de forma aislada la corrección de la lógica criptográfica central del sistema.
- Las pruebas de integración, orientadas a validar el comportamiento del mismo cuando sus componentes interactúan entre sí dentro de un entorno controlado.
- Las pruebas de sistema, enfocadas en evaluar el funcionamiento del software completo desplegado en un entorno de ejecución real.

Cada tipo de prueba se presenta en un epígrafe independiente, donde se describen su fundamento, las herramientas empleadas, la estrategia de evaluación adoptada y los resultados obtenidos. Finalmente, se exponen las conclusiones parciales obtenidas a partir del análisis integral de los resultados de validación.

4.1 Pruebas unitarias

Las pruebas unitarias son un tipo de prueba de software orientadas a verificar el funcionamiento correcto de las partes más pequeñas e independientes del código, comúnmente denominadas unidades. Cada unidad representa una funcionalidad aislada, como un método o un componente específico. En el contexto de un módulo criptográfico, este nivel de prueba resulta especialmente relevante, ya que permite comprobar la exactitud de las operaciones matemáticas y de los mecanismos de seguridad sin interferencias del resto del sistema.

Herramientas utilizadas

Para la implementación de las pruebas unitarias se utilizaron las siguientes herramientas:

- **JUnit 5** constituye el *framework* base para la estructuración y ejecución de todos los casos de prueba. Permite definir métodos de prueba mediante anotaciones (*@Test*) y validar resultados con aserciones como *assertEquals*, *assertThrows* y *assertArrayEquals*.
- **Mockito** es la biblioteca utilizada para la simulación (*mocking*) de dependencias externas. Permite crear objetos simulados que reproducen el comportamiento de repositorios y servicios sin necesidad de ejecutar la lógica real del sistema, lo que garantiza el aislamiento de la unidad bajo prueba.

Ninguno de los conjuntos de pruebas unitarias requiere el contexto de *Spring* para su ejecución; las dependencias se inyectan directamente mediante *ReflectionTestUtils* o se simulan con *Mockito*, lo que garantiza rapidez y aislamiento durante la ejecución.

Estrategia de pruebas

Las pruebas unitarias del proyecto se organizan en tres grupos, cada uno orientado a un servicio crítico distinto: el servicio de cifrado de claves (*KeyEncryptionService*), el servicio de firma digital (*SignatureService*) y el servicio de gestión del ciclo de vida de claves (*KeyService*). La cobertura se definió en función de los aspectos de mayor riesgo criptográfico: la exactitud del cifrado *AES-256-GCM* aplicado a las claves privadas en reposo, la corrección y la interoperabilidad de los algoritmos de firma *RSA* y *ECDSA*, y la integridad de las transiciones de estado del ciclo de vida de claves.

Módulo de cifrado AES-256-GCM

La clase *KeyEncryptionServiceTest* contiene 7 pruebas unitarias orientadas a verificar el comportamiento del servicio responsable del cifrado y descifrado de claves privadas en reposo mediante *AES-256-GCM*. Los aspectos evaluados incluyen la correcta inversión de los procesos de cifrado y descifrado, la generación de vectores de inicialización (IV) aleatorios en cada operación, la detección de manipulaciones mediante la etiqueta de autenticación de *GCM*, el rechazo de datos adicionales de autenticación (AAD) inválidos y la validación de la clave de cifrado de claves (*KEK*) durante la inicialización del servicio.

Los casos de prueba que materializan esta estrategia de validación se presentan en la Tabla 3, donde se detallan las funcionalidades evaluadas y los resultados esperados para cada escenario considerado.

Tabla 3. Pruebas unitarias. Módulo de cifrado AES-256-GCM

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>cifradoYDescifradoSonInversos()</i>	Cifrar una clave privada <i>RSA</i> y descifrarla con el mismo <i>AAD</i> produce los bytes originales.	Los bytes descifrados son idénticos a los originales.
<i>blobCifradoNoEsBase64PuroDeClaveRSA()</i>	El <i>blob</i> almacenado en base de datos no revela la estructura <i>PKCS#8</i> de la clave privada.	El <i>blob</i> cifrado no comienza con “MI”.
<i>cadaCifradoProduceBlobDistintoPorIVAleatorio()</i>	Dos cifrados del mismo dato con igual <i>AAD</i> producen <i>blobs</i> distintos por el <i>IV</i> aleatorio por operación.	Los dos <i>blobs</i> generados son diferentes.
<i>aadIncorrectoFallaAlDescifrar()</i>	Si el <i>AAD</i> al descifrar no coincide con el <i>AAD</i> usado al cifrar, la operación falla.	Se lanza una excepción al descifrar con <i>AAD</i> incorrecto.
<i>blobManipuladoFallaAlDescifrar()</i>	Si un byte del <i>blob</i> cifrado es alterado, la etiqueta de autenticación de <i>GCM</i> detecta la manipulación.	Se lanza una excepción al descifrar el <i>blob</i> manipulado.
<i>kekNoConfiguradoFallaAlIniciar()</i>	Si <i>HSM_MASTER_KEY</i> está vacío, el servicio lanza excepción en la fase <i>@PostConstruct</i> .	Se lanza <i>IllegalStateException</i> al iniciar.
<i>kekDeTamanoIncorrectoFallaAlIniciar()</i>	Si el <i>KEK</i> tiene menos de 32 bytes, el servicio lanza excepción en la fase <i>@PostConstruct</i> .	Se lanza <i>IllegalStateException</i> al iniciar.

Módulo de firma digital

La clase *SignatureServiceTest* contiene 20 pruebas unitarias que evalúan el comportamiento del servicio de firma y verificación de *hashes* precomputados. Esta clase genera pares de claves *RSA*-2048 y *EC P*-256/*P*-384 mediante *JCA* para los casos de prueba, sin depender de claves almacenadas en base de datos. Las pruebas cubren los algoritmos *RSA* con *SHA*-256, *SHA*-384, *SHA*-512, así como los algoritmos *ECDSA* (*P*-256, *P*-384). De manera similar, se evalúa la interoperabilidad de las firmas con verificadores externos estándar, las

validaciones de negocio sobre el estado y tipo de clave, y la zeroización de la clave privada tras cada operación de firma.

Las Tablas 4, 5 y 6 presentan los casos de prueba diseñados para validar el servicio de firma digital, organizados según los algoritmos criptográficos evaluados y las validaciones de negocio asociadas, con sus correspondientes funcionalidades y resultados esperados.

Tabla 4. Pruebas unitarias. Módulo de firma digital, RSA

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>rsaSHA256_signAndVerify_roundTrip()</i>	<i>Round-trip SHA256withRSA: firmar y verificar con la misma clave.</i>	<i>valid=true</i>
<i>rsaSHA384_signAndVerify_roundTrip()</i>	<i>Round-trip SHA384withRSA: firmar y verificar con la misma clave.</i>	<i>valid=true</i>
<i>rsaSHA512_signAndVerify_roundTrip()</i>	<i>Round-trip SHA512withRSA: firmar y verificar con la misma clave.</i>	<i>valid=true</i>
<i>rsa_wrongHashInVerify_returnsInvalid()</i>	Verificar con un <i>hash</i> distinto al firmado.	<i>valid=false</i>
<i>rsa_tamperedSignature_returnsInvalid()</i>	Verificar una firma con un byte alterado.	<i>valid=false</i>
<i>rsa_signatureIsStandardCompatible()</i>	La firma producida por el <i>HSM</i> es interoperable con verificadores externos estándar.	La verificación externa resulta válida.

Tabla 5. Pruebas unitarias. Módulo de firma digital, ECDSA

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>ecdsaP256_SHA256_signAndVerify_roundTrip()</i>	<i>Round-trip SHA256withECDSA con curva P-256.</i>	<i>valid=true</i>
<i>ecdsaP384_SHA384_signAndVerify_roundTrip()</i>	<i>Round-trip SHA384withECDSA con curva P-384.</i>	<i>valid=true</i>

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>ecdsa_wrongHashInVerify_returnsInvalid()</i>	Verificar <i>ECDSA</i> con un <i>hash</i> distinto al firmado.	<i>valid=false</i>
<i>ecdsa_tamperedSignature_returnsInvalid()</i>	Verificar una firma <i>ECDSA</i> con un <i>byte</i> alterado.	<i>valid=false</i> o excepción por <i>DER</i> inválido.

Tabla 6. Pruebas unitarias. Módulo de firma digital, validaciones de negocio

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>sign_keyNotActive_throwsException()</i>	Intentar firmar con una clave en estado <i>REVOKED</i> .	<i>IllegalArgumentExcepti on</i>
<i>sign_keyDestroyed_throwsException()</i>	Intentar firmar con una clave en estado <i>DESTROYED</i> .	<i>IllegalArgumentExcepti on</i>
<i>sign_keyTypeEncryption_throwsException()</i>	Intentar firmar con una clave de tipo <i>ENCRYPTION</i> .	<i>IllegalArgumentExcepti on</i>
<i>sign_algorithmMismatch_RSAkeyECDSAalgo_throwsException()</i>	Usar una clave <i>RSA</i> con un algoritmo <i>ECDSA</i> .	<i>IllegalArgumentExcepti on "Algorithm mismatch"</i>
<i>sign_algorithmMismatch_ECkeyRSAalgo_throwsException()</i>	Usar una clave <i>EC</i> con un algoritmo <i>RSA</i> .	<i>IllegalArgumentExcepti on "Algorithm mismatch"</i>
<i>sign_hashLengthMismatch_throwsException()</i>	Enviar un <i>hash</i> de 16 bytes cuando <i>SHA256withRSA</i> espera 32.	<i>IllegalArgumentExcepti on</i>
<i>sign_invalidBase64Hash_throwsException()</i>	Enviar un <i>string</i> no válido en <i>Base64</i> como <i>hash</i> .	<i>IllegalArgumentExcepti on</i>
<i>verify_keyDestroyed_throwsException()</i>	Verificar una firma con una clave en estado <i>DESTROYED</i> .	<i>IllegalArgumentExcepti on</i>
<i>verify_keyRevoked_allowsVerification()</i>	Verificar una firma con una clave en estado <i>REVOKED</i> .	La verificación procede con éxito.
<i>sign_privateKeysZeroizedAfterUse()</i>	Tras firmar, el <i>array</i> de bytes de la clave privada queda sobrescrito con ceros.	El <i>array</i> contiene únicamente ceros.

Módulo de ciclo de vida de claves

La clase *KeyServiceValidationTest* contiene 15 pruebas unitarias que evalúan las validaciones de creación, rotación y destrucción de claves en el servicio *KeyService*. En esta clase se simula el repositorio y el servicio de cifrado, mientras que la generación de pares de claves se realiza mediante *JCA*. Los aspectos evaluados incluyen la aceptación y el rechazo de tamaños de clave según los límites mínimos establecidos para *RSA* y *EC*, la gestión de alias duplicados con distinción por propietario, la correcta transición de estados durante la rotación y la zeroización del *blob* cifrado al destruir una clave.

En la Tabla 7 se detallan las pruebas unitarias asociadas al ciclo de vida de claves en el servicio *KeyService*, en la que se exponen los casos evaluados y los resultados esperados durante su ejecución.

Tabla 7. Pruebas unitarias. Ciclo de vida de claves

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>createRSA_3072_isAccepted()</i>	Crear una clave <i>RSA</i> de 3072 bits.	Operación exitosa.
<i>createRSA_4096_isAccepted()</i>	Crear una clave <i>RSA</i> de 4096 bits.	Operación exitosa.
<i>createRSA_2048_isRejected()</i>	Intentar crear una clave <i>RSA</i> de 2048 bits, por debajo del mínimo permitido.	<i>IllegalArgumentException</i>
<i>createEC_256_isAccepted()</i>	Crear una clave <i>EC</i> de 256 bits (P-256).	Operación exitosa.
<i>createEC_384_isAccepted()</i>	Crear una clave <i>EC</i> de 384 bits (P-384).	Operación exitosa.
<i>createEC_512_isRejected()</i>	Intentar crear una clave <i>EC</i> de 512 bits (curva no soportada).	<i>IllegalArgumentException</i>
<i>createEC_3072_isRejected()</i>	Intentar crear una clave <i>EC</i> con tamaño 3072, inválido para <i>EC</i> .	<i>IllegalArgumentException</i>
<i>createRSA_256_isRejected()</i>	Intentar crear una clave <i>RSA</i> con tamaño 256, inválido para <i>RSA</i> .	<i>IllegalArgumentException</i>

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>createKey_nullKeySize_throwsException()</i>	Intentar crear una clave con tamaño nulo.	<i>IllegalArgumentExce</i> <i>ption</i>
<i>duplicateAlias_sameOwner_isRejected()</i>	Crear dos claves con igual alias para el mismo propietario.	<i>IllegalArgumentExce</i> <i>ption</i>
<i>duplicateAlias_differentOwner_isAccepted()</i>	Dos usuarios distintos crean claves con el mismo alias.	Ambas creaciones resultan exitosas.
<i>rotateKey_createsNewVersionAndRevokesOld()</i>	Rotar una clave activa genera una nueva versión y revoca la anterior.	Nueva clave con <i>version+1</i> en estado <i>ACTIVE</i> ; clave antigua en estado <i>REVOKED</i> .
<i>rotateKey_onRevokedKey_throwsException()</i>	Intentar rotar una clave en estado <i>REVOKED</i> .	<i>IllegalArgumentExce</i> <i>ption</i>
<i>destroyKey_setsStatusAndZeroizesBlob()</i>	Destruir una clave establece el estado <i>DESTROYED</i> y sobrescribe el <i>blob</i> con ceros.	Estado <i>DESTROYED</i> y <i>blob</i> zeroizado.
<i>destroyKey_alreadyDestroyed_throwsException()</i>	Intentar destruir una clave ya en estado <i>DESTROYED</i> .	<i>IllegalArgumentExce</i> <i>ption</i>

Resultados de las pruebas unitarias

La ejecución de las pruebas unitarias se desarrolló en tres iteraciones. En la primera se detectaron errores que impidieron la ejecución completa de la suite de pruebas. Entre ellos, varios objetos de transferencia de datos no podían instanciarse correctamente dentro del entorno de prueba debido a la forma en la que la generación automática de código había configurado sus constructores. De igual forma, se detectó el uso de una operación sobre arreglos de bytes que no está disponible en la versión de Java empleada en el proyecto. Una vez corregidos estos problemas, en la segunda iteración se identificaron fallos adicionales en las pruebas del módulo de firma digital: los casos dirigidos a verificar el rechazo de operaciones realizadas con claves en estados incorrecto

no propagaban adecuadamente las excepciones hacia la capa de pruebas, lo que impedía validar los resultados esperados. Tras ajustar la configuración de manejo de excepciones, las 42 pruebas unitarias se ejecutaron satisfactoriamente en la tercera iteración, tal como se muestra en la Tabla 8 que aparece seguidamente.

Tabla 8. Resultados de las pruebas unitarias por iteración

Iteración	Pruebas ejecutadas	Pruebas fallidas	Causa principal de fallo
1	42	7	Objetos de transferencia de datos no instanciables en el contexto de pruebas; operación sobre arreglos de bytes no disponible en la versión de Java utilizada.
2	42	4	Excepciones no propagadas correctamente hacia la capa de pruebas en los casos de validación de estado de clave.
3	42	0	-

Los resultados obtenidos en la tercera iteración evidencian el correcto funcionamiento de los mecanismos criptográficos y de gestión de claves implementados. En particular, las pruebas del módulo de cifrado aportan evidencia de que la implementación de *AES-256-GCM* satisface los requisitos definidos para la protección de claves privadas en reposo: la utilización de vectores de inicialización aleatorios (IV) produce cifrados diferentes para un mismo dato, la etiqueta de autenticación permite detectar modificaciones no autorizadas del contenido cifrado y el servicio responde de manera controlada ante configuraciones inválidas de la clave de cifrado de claves (*KEK*). Además, las pruebas del módulo de firma validan la corrección criptográfica de los algoritmos *RSA* y *ECDSA* implementados, la interoperabilidad de las firmas generadas con verificadores criptográficos estándar y el cumplimiento del requisito de zeroización de las claves privadas una vez finalizada la operación de firma. De igual forma, las pruebas del ciclo de vida de claves verifican la correcta aplicación de las restricciones de tamaño establecidas para cada tipo de clave, el aislamiento de alias entre propietarios y la ejecución adecuada de

las transiciones de estado asociadas a los procesos de rotación y destrucción con la lógica esperada de acuerdo con las reglas definidas para el sistema.

4.2 Pruebas de integración

Las pruebas de integración se consideran un tipo de prueba de software orientado a verificar el correcto funcionamiento del sistema cuando sus distintos componentes interactúan entre sí. A diferencia de las pruebas unitarias, que evalúan métodos o clases de forma aislada, estas se enfocan en comprobar que los módulos del sistema trabajan de manera coordinada para producir el resultado esperado ante flujos de operación completos. Este nivel de prueba resulta esencial en el contexto del módulo desarrollado, ya que permite detectar errores derivados de la comunicación entre la capa de control, la capa de servicios y la capa de persistencia. Además, verifica la correcta inicialización del entorno de la aplicación, comprobar el comportamiento del sistema de seguridad ante distintos roles y credenciales, y la integridad del registro de auditoría frente a operaciones exitosas y fallidas.

Herramientas utilizadas

Para la ejecución de las pruebas de integración se emplearon las siguientes herramientas:

- **JUnit 5**, descrito anteriormente.
- **Spring Boot Test** proporciona el entorno necesario para las pruebas de integración mediante la carga del contexto completo de la aplicación, incluidos sus componentes, servicios, controladores y configuración de seguridad.
- **MockMvc** permite simular peticiones *HTTP* a los *endpoints* del controlador sin necesidad de desplegar la aplicación en un servidor externo, lo que posibilita validar de forma automatizada el comportamiento de la capa *REST*.
- **Spring Security Test** facilita la simulación de contextos de autenticación mediante el posprocesador *jwt()*, el cual construye *tokens JWT* sintéticos con roles específicos sin necesidad de conectarse a *Keycloak*. Esto permite verificar los accesos protegidos y los comportamientos ante solicitudes sin credenciales o con permisos insuficientes de manera completamente reproducible.

La infraestructura de soporte incluye una base de datos H2 en memoria configurada en modo *PostgreSQL*, la cual reemplaza a *SQL* durante la ejecución de las pruebas. El esquema se crea y se destruye en cada ejecución mediante la estrategia *create-drop*, que garantiza el aislamiento entre casos de prueba. Adicionalmente, se definió la clase *TestJwtDecoderConfig* como configuración suplementaria (*@TestConfiguration*), que provee un *bean JwtDecoder* simulado con prioridad *@Primary*, lo que impide a *Spring Boot* conectarse al servidor *Keycloak* real durante las pruebas.

Estrategia de pruebas

Las pruebas de integración se organizan en dos clases. La primera, *SoftwareHsmApplicationTests*, verifica que el contexto completo de *Spring Boot* se inicializa correctamente y sin errores. La segunda, *KeyControllerIntegrationTest*, compuesta por 26 casos de prueba, evalúa el comportamiento de los *endpoints REST* del sistema en distintos escenarios, agrupados en cinco categorías: creación y consulta de claves, firma y verificación, ciclo de vida de claves, control de acceso y seguridad, y registro de auditoría. Antes de la ejecución de cada caso de prueba se restablece el estado de la base de datos, lo que garantiza el aislamiento entre casos y la reproducibilidad de los resultados.

Prueba de arranque del contexto

La clase *SoftwareHsmApplicationTests* contiene una única prueba cuyo propósito es verificar que el contexto completo de *Spring Boot* se inicia sin errores bajo la configuración de pruebas. En este caso se valida que todos los *beans* del sistema, incluidos los repositorios *JPA* (*Java Persistence API*), los servicios de criptografía y la configuración de seguridad, se inicializan correctamente. Además, verifica que la *KEK* se procesa sin fallos durante la fase *@PostConstruct*.

A continuación, se presenta la Tabla 9, en la que se detalla la prueba de integración correspondiente al arranque del contexto (*SoftwareHsmApplicationTests*), y se exponen la funcionalidad evaluada y el resultado esperado.

Tabla 9. Prueba de integración. Arranque del contexto

Nombre del test	Funcionalidad evaluada	Resultado esperado
<i>contextLoads()</i>	El contexto de <i>Spring Boot</i> arranca sin errores con la configuración de pruebas, incluyendo la inicialización de <i>beans</i> y el procesamiento de la <i>KEK</i> .	El contexto se levanta sin excepciones.

Pruebas de flujos funcionales de la *API REST*

La clase *KeyControllerIntegrationTest* concentra las pruebas orientadas a validar el comportamiento funcional de los *endpoints REST* del sistema en escenarios de operación completos. Desde la Tabla 10 hasta la 14 se presentan los detalles de las funcionalidades evaluadas y los resultados esperados correspondientes a cada grupo de pruebas.

Tabla 10. Pruebas de integración. Creación y consulta de claves

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>POST_createKey_RSA_returns201_withMetadata()</i>	<i>POST /api/keys</i>	Crear clave <i>RSA</i> con datos válidos y usuario autenticado.	201 <i>Created</i> con <i>keyId</i> , <i>alias</i> , <i>publicKey</i> y <i>status=ACTIVE</i> .
<i>POST_createKey_EC_returns201_withMetadata()</i>	<i>POST /api/keys</i>	Crear clave <i>EC</i> con datos válidos.	201 <i>Created</i> con <i>algorithm=EC</i> .
<i>GET_keyMetadata_returns200()</i>	<i>GET /api/keys/{id}</i>	Crear una clave y luego consultar sus metadatos.	200 OK con campos correctos.
<i>GET_keyMetadata_otherUsersKey_returns400()</i>	<i>GET /api/keys/{id}</i>	Usuario B intenta obtener metadatos de la clave de usuario A.	Error de recurso no encontrado (404 <i>Not Found</i>) la existencia del recurso no se revela al no propietario.

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>GET_listKeys_returnsOnlyOwnKeys()</i>	<i>GET /api/keys</i>	Listar claves devuelve únicamente las pertenecientes al usuario autenticado.	Solo aparecen las claves propias en el resultado.
<i>GET_publicKey_returns200_withPEM()</i>	<i>GET /api/keys/{id}/public</i>	Obtener la clave pública de una clave propia.	200 OK con bloque <i>PEM</i> que contiene <i>"BEGIN PUBLIC KEY"</i> .
<i>GET_publicKey_byAnyAuthenticatedUser_returns200()</i>	<i>GET /api/keys/{id}/public</i>	Cualquier usuario autenticado puede obtener la clave pública, independientemente del propietario.	200 OK.

Tabla 11. Pruebas de integración. Firma y verificación

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>signAndVerify_RSA_SHA256_fullFlow()</i>	<i>POST /api/keys/{id}/sign,</i> <i>POST /api/keys/{id}/verify</i>	Crear clave <i>RSA</i> , firmar un <i>hash</i> , verificar la firma.	Firma exitosa y verificación con <i>valid=true</i> .
<i>signAndVerify_ECDSA_SHA256_fullFlow()</i>	<i>POST /api/keys/{id}/sign,</i> <i>POST /api/keys/{id}/verify</i>	Crear clave <i>EC</i> , firmar un <i>hash</i> , verificar la firma.	Firma exitosa y verificación con <i>valid=true</i> .
<i>sign_wrongAlgorithmForKeyType_returns400()</i>	<i>POST /api/keys/{id}/sign</i>	Crear clave <i>RSA</i> , intentar firmar con un algoritmo incompatible con ese tipo de clave.	Error de solicitud incorrecta (<i>400 Bad Request</i>) con indicación de incompatibilidad de algoritmo.
<i>sign_otherUsersKey_returns400()</i>	<i>POST /api/keys/{id}/sign</i>	Usuario B intenta firmar con la clave de usuario A.	Error de solicitud incorrecta (<i>400 Bad Request</i>) por

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
			violación del principio de titularidad.
<i>sign_adminCannotSignOtherUsersKey()</i>	<i>POST</i> <i>/api/keys/{id}/sign</i>	El rol administrador intenta firmar con la clave de un usuario.	Error de solicitud incorrecta (<i>400 Bad Request</i>) confirma que el principio de titularidad aplica a todos los roles.
<i>verify_withRevokedKey_returns200_valid()</i>	<i>POST</i> <i>/api/keys/{id}/verify</i>	Crear clave, firmar, revocar la clave y verificar la firma con ella.	Respuesta satisfactoria (<i>200 OK</i>) con resultado de verificación positivo.
<i>verify_withDestroyedKey_returns400()</i>	<i>POST</i> <i>/api/keys/{id}/verify</i>	Crear clave, firmar, destruir la clave e intentar verificar.	Error de solicitud incorrecta (<i>400 Bad Request</i>) confirma que la destrucción de la clave impide toda operación.

Tabla 12. Pruebas de integración. Ciclo de vida de claves

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>rotateKey_oldKeyRevoked_newKeyActive()</i>	<i>POST</i> <i>/api/keys/{id}/rotate</i>	Rotar una clave activa genera una nueva versión activa y revoca la anterior.	Clave antigua en <i>REVOKED</i> , nueva clave en <i>ACTIVE</i> con <i>version=2</i> .
<i>destroyKey_statusIsDestroyed()</i>	<i>DELETE</i> <i>/api/keys/{id}</i>	Destruir una clave activa establece su estado a <i>DESTROYED</i> .	Estado <i>DESTROYED</i> en la respuesta.
<i>sign_afterDestroy_returns400()</i>	<i>POST</i> <i>/api/keys/{id}/sign</i>	Destruir una clave e intentar firmar con ella.	Error de solicitud incorrecta (<i>400 Bad Request</i>).

Tabla 13. Pruebas de integración. Control de acceso y seguridad

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>request_withoutToken_returns401()</i>	Cualquier <i>endpoint</i>	Enviar una solicitud sin <i>token JWT</i> .	Error de autenticación requerida (401 <i>Unauthorized</i>).
<i>auditor_accessKeyEndpoint_returns403()</i>	<i>POST /api/keys</i>	El rol auditor intenta crear una clave.	Error de acceso prohibido (403 <i>Forbidden</i>) por restricción de rol.
<i>auditor_accessAuditLogs_returns200()</i>	<i>GET /api/audit/logs</i>	El rol auditor accede al registro de auditoría.	Respuesta satisfactoria (200 OK).
<i>admin_accessAuditLogs_returns403()</i>	<i>GET /api/audit/logs</i>	El rol administrador intenta acceder al registro de auditoría.	Error de acceso prohibido (403 <i>Forbidden</i>) confirma la separación de funciones entre administrador y auditor.
<i>user_accessAuditLogs_returns403()</i>	<i>GET /api/audit/logs</i>	El rol usuario intenta acceder al registro de auditoría.	Error de acceso prohibido (403 <i>Forbidden</i>) por restricción de rol.

Tabla 14. Pruebas de integración. Registro de auditoría

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
<i>signOperation_createsAuditLogEntry()</i>	<i>POST /api/keys/{id}/sign</i>	Tras una firma exitosa, existe un registro en <i>audit_logs</i> con <i>operation=SIGN</i> y <i>result=SUCCESS</i> .	Registro de auditoría presente con los campos correctos.
<i>failedSign_createAuditLogWithFailure()</i>	<i>POST /api/keys/{id}/sign</i>	Intentar firmar con una clave <i>REVOKED</i> genera un registro con <i>result=FAILURE</i> .	Registro de auditoría presente con <i>result=FAILURE</i> .
<i>GET_auditLogById_returns200()</i>	<i>GET /api/audit/logs/{id}</i>	Obtener un registro de auditoría concreto	200 OK con el registro correcto.

Nombre del test	Endpoint evaluado	Descripción	Resultado esperado
		por su identificador.	
<i>GET_auditLogById_nonExistent_returns404()</i>	<i>GET /api/audit/logs/{id}</i>	Solicitar un registro de auditoría con identificador inexistente.	Error de recurso no encontrado (<i>404 Not Found</i>).

Resultados de las pruebas de integración

La validación mediante pruebas de integración requirió dos iteraciones de ejecución y corrección antes de alcanzar resultados satisfactorios. En la primera, se detectaron errores que impidieron la ejecución correcta de varios casos: una clase auxiliar de configuración de pruebas fue declarada con una anotación de propósito general en lugar de la anotación específica para entornos de prueba, lo que provocaba que el contenedor de *Spring Boot* no cargara correctamente los componentes de acceso a datos ni los componentes de serialización. Además, varias pruebas del flujo de rotación de claves esperaban un código de respuesta que no correspondía al que el servidor devuelve para ese tipo de operación. Estos problemas fueron corregidos en la segunda iteración, en la que la totalidad de las 27 pruebas se ejecutó de forma exitosa. En la Tabla 15 se pueden observar los resultados obtenidos.

Tabla 15. Resultados de las pruebas de integración por iteración

Iteración	Pruebas ejecutadas	Pruebas fallidas	Causa principal
1	27	9	Configuración de pruebas declarada con anotación incorrecta, lo que impide la carga de componentes de acceso a datos y serialización; respuesta esperada incorrecta en pruebas del flujo de rotación de claves.
2	27	0	-

Los resultados de la segunda iteración confirman que el ciclo de vida completo de las claves, los flujos de firma y verificación, las políticas de control de acceso por roles y el registro de auditoría operan correctamente a través de la *API REST*. Las solicitudes sin *token JWT* recibieron un error de autenticación requerida en todos los *endpoints* protegidos. El principio de titularidad fue validado en múltiples escenarios, sin que ningún usuario pudiera operar sobre claves ajenas, incluso con rol administrador. El sistema responde con un error de recurso no encontrado ante accesos a recursos no propios, lo que evita revelar su existencia. El registro de auditoría capturó tanto las operaciones exitosas como las fallidas con los campos esperados.

4.3 Pruebas del sistema

Las pruebas de sistema son un tipo de prueba de software orientadas a evaluar el comportamiento del sistema completo en un entorno de ejecución real, con todos sus componentes activos e integrados. A diferencia de las pruebas unitarias y de integración, que operan sobre clases aisladas o contextos simulados respectivamente, las pruebas de sistema verifican que el software funciona correctamente cuando es ejecutado tal como lo haría en un entorno de producción: con la base de datos real, el servidor de autenticación real y la *API* desplegada y accesible desde un cliente externo. Esta distinción es fundamental, ya que aspectos como la emisión y validación de *tokens JWT* reales por parte de *Keycloak*, la persistencia efectiva de datos en *PostgreSQL* o la comunicación entre contenedores solo pueden ser comprobados a este nivel.

Herramientas utilizadas

Para la realización de las pruebas de sistema se emplearon las siguientes herramientas:

- **Docker Compose** se utilizó para el despliegue del entorno de ejecución completo, que levanta de forma coordinada los tres contenedores que conforman el sistema: el módulo *HSM (Spring Boot)*, la base de datos *PostgreSQL* y el servidor de autenticación *Keycloak*.
- **Streamlit** es una biblioteca de *Python* orientada al desarrollo simplificado de interfaces *web*. Se utilizó para construir un cliente *web* que permite interactuar con la totalidad de las funcionalidades expuestas por la *API REST* del *HSM* desde un navegador, que incluye la autenticación, la gestión del ciclo de vida de claves, la firma digital

de documentos PDF, la verificación de firmas y la consulta del registro de auditoría.

Estrategia de pruebas

El cliente *web* desarrollado con *Streamlit* actuó como instrumento de prueba para ejecutar escenarios funcionales de extremo a extremo sobre el sistema completo desplegado. Los escenarios evaluados cubrieron la autenticación y gestión de sesión con *Keycloak*, la gestión del ciclo de vida de claves, la firma digital de documentos PDF y la verificación de firmas, el control de acceso por roles, y la consulta del registro de auditoría. El objetivo fue verificar que la interacción entre todos los componentes del sistema produce resultados coherentes: que *Keycloak* emite *tokens* válidos aceptados por el *HSM*, que las operaciones criptográficas persisten correctamente en *PostgreSQL*, y que el registro de auditoría refleja fielmente todas las actividades del sistema.

Los escenarios funcionales validados a través de este cliente y los resultados esperados en cada caso se recogen en la Tabla 16 que aparece a continuación.

Tabla 16. Prueba de extremo a extremo. Cliente *web Streamlit*

Escenario evaluado	Descripción	Resultado esperado
Autenticación con <i>Keycloak</i>	Inicio de sesión con credenciales válidas, obtención del <i>JWT</i> y carga de la interfaz según el rol del usuario.	Autenticación exitosa, <i>token</i> válido, navegación disponible según rol.
Expiración automática de sesión	Esperar la expiración del <i>token JWT</i> y verificar que el sistema detecta la situación y cierra la sesión.	El sistema detecta el <i>token</i> expirado y redirige al formulario de <i>login</i> .
Creación de clave criptográfica	Crear un par de claves <i>RSA</i> con alias, tamaño y tipo especificados.	Respuesta 201 con el identificador y clave pública generados; el par de claves queda registrado en la base de datos.
Listado y filtrado de claves	Consultar las claves del usuario con filtros de estado.	Solo se muestran las claves pertenecientes al usuario autenticado, con sus

Escenario evaluado	Descripción	Resultado esperado
		estados reflejados correctamente.
Rotación de clave con confirmación	Rotar una clave activa y confirmar la operación en la interfaz.	Clave anterior en estado <i>REVOKED</i> , nueva clave activa con versión incrementada.
Destrucción de clave con confirmación explícita	Destruir una clave e introducir la cadena de confirmación requerida.	Clave en estado <i>DESTROYED</i> ; las operaciones posteriores sobre ella son rechazadas.
Firma digital de documento PDF	Subir un PDF, seleccionar algoritmo y clave, firmar y descargar el archivo <i>.sig</i> .	Firma generada correctamente en <i>Base64</i> y descargada como archivo; el documento original no es modificado.
Verificación de firma válida	Subir el mismo PDF y el archivo <i>.sig</i> obtenido y verificar la firma.	Resultado FIRMA VÁLIDA.
Verificación con documento modificado	Alterar el contenido del PDF y verificar la misma firma.	Resultado FIRMA INVÁLIDA, alteración detectada.
Control de acceso por rol en la interfaz	Intentar acceder a funcionalidades no autorizadas para el rol del usuario activo.	Las secciones restringidas no son accesibles y las solicitudes a <i>endpoints</i> no autorizados son rechazadas con 403.
Consulta del registro de auditoría	Acceder al registro de auditoría con un usuario de rol auditor, aplicar filtros y verificar que las operaciones anteriores quedaron registradas.	Los registros de las operaciones ejecutadas aparecen en el historial con los datos correctos de operación, usuario y resultado.

Resultados

Las pruebas de sistema se ejecutaron en una única iteración sin detectar errores. Todos los escenarios evaluados produjeron los resultados esperados, lo cual confirma la adecuada integración entre los tres componentes del sistema, el módulo *HSM*, *PostgreSQL* y *Keycloak*, en un entorno de ejecución real. La

autenticación mediante *Keycloak* generó *tokens* válidos que fueron aceptados por el *HSM* en todos los casos; asimismo, las operaciones criptográficas persistieron correctamente en *PostgreSQL* junto con los metadatos correspondientes, y el registro de auditoría reflejó de manera consistente las actividades realizadas en el sistema. Adicionalmente, el mecanismo de verificación de firmas demostró su capacidad para detectar alteraciones en los documentos, al producir un resultado negativo cuando el contenido del PDF era modificado después de la firma.

La Tabla 17 resume los resultados alcanzados durante la ejecución de las pruebas de sistema en los escenarios evaluados.

Tabla 17. Resultados de las pruebas del sistema por iteración

Iteración	Escenarios ejecutados	Escenarios fallidos	Causa principal
1	11	0	-

4.4 Conclusiones parciales

A partir de los resultados de la validación presentados en este capítulo se puede concluir que:

1. Las pruebas unitarias, ejecutadas en tres iteraciones, validan el correcto funcionamiento de los componentes criptográficos del sistema de forma aislada, lo que evidencia la adecuada implementación de los mecanismos de cifrado *AES-256-GCM*, de los algoritmos de firma *RSA* y *ECDSA*, y de las validaciones del ciclo de vida de claves. Los errores detectados en la primera iteración correspondieron a inconsistencias de compilación que fueron corregidas antes de alcanzar resultados satisfactorios.
2. Las pruebas de integración, completadas en dos iteraciones, confirman la coherencia funcional de los flujos *HTTP* del sistema, lo que garantiza la correcta interacción entre los módulos de control, servicios y persistencia, así como el cumplimiento de las políticas de control de acceso por roles, el principio de titularidad de la clave y la integridad del registro de auditoría.
3. Las pruebas de sistema, realizadas mediante el cliente web desplegado en un entorno real con *Docker Compose*, demostraron el correcto funcionamiento e integración del módulo *HSM* con *PostgreSQL* y

Keycloak. Asimismo, la ejecución satisfactoria de las pruebas en una única iteración validó los flujos de autenticación, gestión de claves, firma digital y auditoría en condiciones equivalentes a las de un entorno de producción.

4. El proceso de validación en su conjunto confirma el cumplimiento de los requisitos funcionales y de seguridad definidos para el módulo, lo que asegura la confiabilidad del sistema en sus tres dimensiones evaluadas: corrección criptográfica, seguridad de acceso e integridad operacional.

CONCLUSIONES

A continuación, se presentan las conclusiones derivadas del análisis y cumplimiento de los objetivos específicos del presente trabajo, las cuales sintetizan los principales resultados obtenidos en cada etapa del estudio.

- 1 El análisis de los fundamentos teóricos, normativos y tecnológicos permitió establecer el marco conceptual, regulatorio y tecnológico que sustenta el desarrollo de un módulo de seguridad por software para la gestión de claves criptográficas y la firma digital, además de determinar las tecnologías y estándares necesarios para su implementación.
- 2 Se definió un módulo de seguridad por software sustentado en una arquitectura multicapa, que incorpora una *API REST*, un modelo de datos estructurado y un mecanismo de control de acceso basado en roles, que garantizó niveles adecuados de seguridad, mantenibilidad e interoperabilidad.
- 3 A partir del diseño arquitectónico se implementó un sistema criptográfico operativo que integra la gestión del ciclo de vida de claves, la firma digital y un esquema de auditoría segura. La solución incorpora mecanismos de generación, rotación, revocación y destrucción segura de claves criptográficas; protección de claves privadas en reposo mediante cifrado autenticado; operaciones de firma y verificación digital y un registro de auditoría inmutable que garantiza la trazabilidad de las operaciones criptográficas realizadas.
- 4 La validación del módulo mediante pruebas unitarias, de integración y de sistema permitió verificar el correcto funcionamiento de los mecanismos criptográficos, la estabilidad operativa de los flujos principales y el cumplimiento de los requisitos funcionales y de seguridad.

RECOMENDACIONES

- 1 Incorporar autenticación de dos factores mediante contraseña de un solo uso (*OTP/TOTP*) para operaciones críticas como la firma digital, de forma tal que se añada una capa adicional de seguridad más allá de la autenticación por *token*.
- 2 Evaluar el rendimiento y la escalabilidad del sistema mediante la ejecución de pruebas de carga y estrés, con el objetivo de determinar los límites operacionales del módulo, la latencia de las operaciones criptográficas bajo condiciones de alta concurrencia y su comportamiento en entornos de producción.

REFERENCIAS BIBLIOGRÁFICAS

- [1] K. W. Chong, Y. S. Kim, and J. Choi, "A study of factors affecting intention to adopt a cloud-based digital signature service," *Information*, vol. 12, no. 2, p. 60, 2021.
- [2] C. K. Yee and M. F. Zolkipli, "Review on confidentiality, integrity and availability in information security," *Journal of ICT in Education*, vol. 8, no. 2, pp. 34-42, 2021.
- [3] L. Mayr, G. Zambonin, F. Schardong, and R. Custódio, "One-time certificates for reliable and secure document signing," *arXiv preprint arXiv:2208.03951*, 2022.
- [4] Y. Aref, Z. Alfughi, and A. Ouda, "Evaluating hardware security modules: Key factors for choosing the best option for organizational security requirements," presented at the 2025 International Conference on Smart Applications, Communications and Networking (SmartNets), 2025.
- [5] National Institute of Standards and Technology, "Security Requirements for Cryptographic Modules," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, FIPS PUB 140-3, 2019.
- [6] J. T. Amael, J. E. Istiyanto, and O. Natan, "Enhancing Industrial Cybersecurity: SoftHSM Implementation on SBCs for Mitigating MITM Attacks," *arXiv preprint arXiv:2409.09948*, 2024.
- [7] Y. Aref and A. Ouda, "HSM4SSL: Leveraging hsms for enhanced intra-domain security," *Future Internet*, vol. 16, no. 5, p. 148, 2024.
- [8] Ministerio del Interior de la República de Cuba, "Resolución No. 23/2022 sobre el uso de la firma electrónica y los servicios de certificación electrónica," *Gaceta Oficial de la República de Cuba*, 2022.
- [9] M. Dahlmanns, C. Sander, R. Decker, and K. Wehrle, "Secrets revealed in container images: an internet-wide study on occurrence and impact," in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, 2023, pp. 797-811.
- [10] I. Aciobăniței, Ș.-C. Arseni, E. Bureacă, and M. Togan, "A comprehensive and privacy-aware approach for remote qualified electronic signatures," (in English), *Electronics*, vol. 13, no. 4, p. 757, 2024.

- [11] S. Rana, F. K. Parast, B. Kelly, Y. Wang, and K. B. Kent, "A comprehensive survey of cryptography key management systems," *Journal of Information Security and Applications*, vol. 78, p. 103607, 2023.
- [12] W. Yeoh, M. Liu, M. Shore, and F. Jiang, "Zero trust cybersecurity: critical success factors and a maturity assessment framework," *Computers & Security*, vol. 133, p. 103412, 2023.
- [13] M. D. Nieves, D. Kelley, and V. Y. Pillitteri, "An Introduction to Information Security," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Special Publication SP 800-12 Rev. 1, 2017, Accessed on: May 22, 2026.
- [14] M. Unigwe, "The views of information security professionals toward information security objectives: confidentiality, integrity, and availability triad," Doctoral dissertation, Trident University International, 2022.
- [15] M. S. Al Reshan, "IoT-based application of information security triad," (in English), *International Journal of Computer Science & Network Security*, pp. 85-92, 2024.
- [16] A. Vishvanath and D. Ganesh, "Cloud Safe: A Survey of Encryption, Access Control, and Network Protection Strategies," (in English), *International Journal of Electrical and Electronics Engineering*, vol. 11, no. 12, pp. 208-209, 2024.
- [17] E. Barker, "Recommendation for key management: Part 1 – General," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Technical Report (Government Report / NIST Special Publication) NIST SP 800-57 Part 1 Rev. 5, 2020, Available: <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>, Accessed on: May 15, 2026.
- [18] A. Gomes, C. Santos, C. Wanzeller, and P. Martins, "Database encryption for balance between performance and security," *J. Inf. Assur. Cybersec*, pp. 1-9, 2021.
- [19] G. Alagic *et al.*, "Status report on the third round of the NIST post-quantum cryptography standardization process," National Institute of Standards and Technology (NIST), 2022.

- [20] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," National Institute of Standards and Technology, Gaithersburg, MD, USA, 2015.
- [21] J. A. Khan, "Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) " in *Improving security, privacy, and trust in cloud computing*: IGI Global Scientific Publishing, 2024, pp. 113-126.
- [22] M. Penelova, "Access control models," *Cybernetics and Information Technologies*, vol. 21, no. 4, pp. 77-104, 2021.
- [23] N. Sakimura, J. Bradley, M. Jones, B. d. Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation, 2014.
- [24] J. Primbs and M. Menth, "OIDC²: Open Identity Certification With OpenID Connect," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 1880-1898, 2024.
- [25] European Parliament and Council of the European Union, "Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC," in "910/2014," Official Journal of the European Union, Brussels, Belgium(EU) No 910/2014, 2014, vol. (EU) No 910/2014 Available: <https://eur-lex.europa.eu/eli/reg/2014/910/oj>, Accessed on: May 15, 2026.
- [26] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," in "Federal Information Processing Standards Publication," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, , Standard FIPS PUB 186-5, 2023, Available: <https://doi.org/10.6028/NIST.FIPS.186-5>.
- [27] S. Aggarwal and N. Kumar, "Digital signatures," in *Advances in computers*, vol. 121: Elsevier, 2021, pp. 95-107.
- [28] N. M. Mukhammadovich and A. R. Djuraevich, "Working with cryptographic key information," *International Journal of Electrical and Computer Engineering*, vol. 13, no. 1, p. 911, 2023.
- [29] W. Lin, "Digital signature," in *Trends in data protection and encryption technologies*: Springer, 2023, pp. 77-81.

- [30] European Parliament and Council of the European Union, "Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework," Official Journal of the European Union, Brussels, Belgium 2024/1183, 2024, vol. (EU) 2024/1183 Available: <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>, Accessed on: May 15, 2026.
- [31] European Telecommunications Standards Institute (ETSI), "Electronic Signatures and Infrastructures (ESI); CAdES digital signatures; Part 1: Building blocks and CAdES baseline signatures," European Telecommunications Standards Institute (ETSI), Sophia Antipolis, France, Technical Standard EN 319 122-1 V1.3.0, 2021, Available: https://www.etsi.org/deliver/etsi_en/319100_319199/31912201/01.03.00_60/en_31912201v010300p.pdf, Accessed on: May 15, 2026.
- [32] International Organization for Standardization, "ISO/IEC 19790:2025. Information Technology. Security Techniques. Security Requirements for Cryptographic Modules," in "ISO/IEC Standards," International Organization for Standardization, Geneva, Switzerland, International Standard 2025.
- [33] M. Sommerhalder, "Hardware security module," in *Trends in Data Protection and Encryption Technologies*: Springer, 2023, pp. 83-87.
- [34] V. Mulder, A. Mermoud, V. Lenders, and B. Tellenbach, *Trends in data protection and encryption technologies*. Springer, 2023.
- [35] S. Shaikh and T. P. Fizarido, "HSM and TPM Failures in Cloud: A Real-World Taxonomy and Emerging Defenses," *arXiv preprint arXiv:2507.17655*, 2025.
- [36] S. Goldschmidt, "Suitability of Hardware Security Modules for Securing Remote Access in High-Performance Environments," Diploma thesis, Technische Universität Wien, 2025.
- [37] S. Li *et al.*, "Post-quantum security: Opportunities and challenges," *Sensors*, vol. 23, no. 21, p. 8744, 2023.
- [38] OASIS Open, "Key Management Interoperability Protocol Specification Version 2.1," in "OASIS Standard," OASIS Open, Technical Standard May 2020, Available: <https://docs.oasis-open.org/kmip/kmip-spec/v2.1/os/kmip-spec-v2.1-os.html>, Accessed on: May 15, 2026.

- [39] HashiCorp, "Vault KMIP Secrets Engine," Accessed on: May 15, 2026. Online documentation. Available: <https://developer.hashicorp.com/vault/docs/secrets/kmip>
- [40] HashiCorp, "KMIP Secrets Engine API," Accessed on: May 15, 2026. Online documentation. Available: <https://developer.hashicorp.com/vault/api-docs/secret/kmip>
- [41] E. Barker and A. Roginsky, "Transitioning the use of cryptographic algorithms and key lengths," National Institute of Standards and Technology, SP 800-131A Rev.2, 2018, Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-131Ar2.pdf>., Accessed on: May 20, 2026.
- [42] Consejo de Estado de la República de Cuba, "Decreto-Ley 79/2023 sobre el desarrollo, aplicación y uso de dispositivos de protección criptográfica y servicios en la esfera de la criptografía," Gaceta Oficial de la República de Cuba, 2023, Available: <https://www.gacetaoficial.gob.cu/es/gaceta-oficial-no-89-ordinaria-de-2024>, Accessed on: May 15, 2026.
- [43] Consejo de Ministros de la República de Cuba, "Decreto 106/2024. Reglamento del Decreto-Ley 79/2023 sobre dispositivos de protección criptográfica y servicios en la esfera de la criptografía," Gaceta Oficial de la República de Cuba, 2024, Available: <https://www.gacetaoficial.gob.cu/es/decreto-106-de-2024-de-consejo-de-ministros>, Accessed on: May 15, 2026.
- [44] R. Focardi and F. L. Luccio, "A formally verified configuration for hardware security modules in the cloud," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 412-428.
- [45] I. Kuzminykh, B. Ghita, and S. Shiaeles, "Comparative analysis of cryptographic key management systems," presented at the International Conference on Next Generation Wired/Wireless Networking, 2020.
- [46] I. Buljic, E. Kadusic, T. Cvijanovic, N. Hadzajlic, and N. Zivic, "Comparative Performance Analysis of Leading Backend Frameworks for Developers," presented at the 2025 24th International Symposium INFOTEH-JAHORINA (INFOTEH), 2025.

- [47] S. Batyrbekova and G. Esenalieva, "Research on Data Security Approaches in Spring Boot for Rest API," (in English), *Preprints.org*, 2025, Art. no. 202501.0908.
- [48] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "OAuth 2.0 security best current practice," IETF Tech Report draft-ietf-oauth-security-topics-16, 2020, Accessed on: May 15, 2026.
- [49] A. Chatterjee and A. Prinz, "Applying spring security framework with KeyCloak-based OAuth2 to protect microservice architecture APIs: a case study," *Sensors*, vol. 22, no. 5, p. 1703, 2022.
- [50] N. Dimitrijević, N. Zdravković, M. Bogdanović, and A. Mesterovic, "Advanced Security Mechanisms in the Spring Framework: JWT, OAuth, LDAP and Keycloak," in *Proceedings of the 14th International Conference on Business Information Security (BISEC 2023)*, 2024, pp. 64-70.
- [51] S. V. Salunke and A. Ouda, "A performance benchmark for the PostgreSQL and MySQL databases," *Future Internet*, vol. 16, no. 10, p. 382, 2024.
- [52] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, "A container security survey: Exploits, attacks, and defenses," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1-36, 2025.
- [53] S. Saroja and S. Haseena, "Functional and non-functional requirements in agile software development," in *Agile software development: trends, Challenges and Applications*, 2023, pp. 71-86.
- [54] S. A. Rukmono, F. Zamfirov, L. Ochoa, F. Pex, and M. R. Chaudron, "An explanation of software architecture explanations," *Empirical Software Engineering*, vol. 31, no. 3, p. 59, 2026.
- [55] Z. Wan, Y. Zhang, X. Xia, Y. Jiang, and D. Lo, "Software architecture in practice: Challenges and opportunities," in *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*, 2023, pp. 1457-1469.

GLOSARIO DE SIGLAS

Sigla	Significado
ABAC	Attribute-Based Access Control
AES	Advanced Encryption Standard
API	Application Programming Interface
CPU	Central Processing Unit
ECDSA	Elliptic Curve Digital Signature Algorithm
eIDAS	Electronic Identification, Authentication and Trust Services
HSM	Hardware Security Module
HTTPS	Hypertext Transfer Protocol Secure
ISO	International Organization for Standardization
JWT	JSON Web Token
JWKS	JSON Web Key Set
KEK	Key Encryption Key
KMIP	Key Management Interoperability Protocol
MVCC	Multi-Version Concurrency Control
NIST	National Institute of Standards and Technology
OAuth 2.0	Open Authorization 2.0
OIDC	OpenID Connect
OTP	One-Time Password
PKCS	Public-Key Cryptography Standards
RBAC	Role-Based Access Control
REST	Representational State Transfer
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TLS	Transport Layer Security
TOTP	Time-Based One-Time Password
UUID	Universally Unique Identifier

GLOSARIO TÉRMINOS

Autenticación: Proceso mediante el cual se verifica la identidad de un usuario, sistema o entidad antes de permitir el acceso a recursos protegidos.

Auditoría: Proceso sistemático de registro, monitoreo y revisión de eventos, actividades y operaciones realizadas en un sistema, con el propósito de verificar el cumplimiento de políticas, detectar incidentes de seguridad y garantizar la trazabilidad de las acciones ejecutadas.

Auditoría inmutable: Mecanismo de auditoría que garantiza que los registros de eventos almacenados no puedan ser modificados, alterados o eliminados sin dejar evidencia de dichas acciones, con el fin de preservar su integridad y valor probatorio.

Autorización: Proceso que determina las acciones o recursos a los que una entidad autenticada puede acceder de acuerdo con las políticas de seguridad establecidas.

Certificado digital: Documento electrónico emitido por una Autoridad de Certificación que vincula la identidad de una persona, organización o sistema con una clave pública, lo que permite verificar firmas digitales y establecer comunicaciones seguras.

Ciclo de vida de claves criptográficas: Conjunto de procesos asociados a la creación, almacenamiento, distribución, uso, rotación, revocación, archivado y destrucción de claves criptográficas.

Cifrado: Proceso criptográfico mediante el cual la información se transforma en un formato ilegible para impedir el acceso no autorizado a su contenido.

Claim: Información o atributo contenido dentro de un *token JWT* que describe características de un usuario, entidad o sesión, tales como identidad, roles, permisos o período de validez.

Control de acceso basado en atributos: Modelo de control de acceso en el que las decisiones de autorización se toman a partir de atributos asociados a usuarios, recursos, acciones o contexto de ejecución.

Control de acceso basado en roles: Modelo de control de acceso que asigna permisos a roles previamente definidos y concede dichos permisos a los usuarios en función de los roles que poseen.

Criptografía: Disciplina que estudia los métodos y técnicas para proteger la confidencialidad, integridad, autenticidad y no repudio de la información.

Criptografía asimétrica: Esquema criptográfico que utiliza un par de claves matemáticamente relacionadas: una clave pública y una clave privada.

Criptografía simétrica: Esquema criptográfico que utiliza una misma clave secreta para cifrar y descifrar información.

Destrucción segura de claves: Procedimiento mediante el cual una clave criptográfica es eliminada de forma irreversible para impedir su recuperación o reutilización.

Endpoint: Punto de acceso específico de una interfaz de programación de aplicaciones (*API*) mediante el cual los clientes pueden invocar operaciones o acceder a recursos expuestos por un servicio.

Firma digital: Mecanismo criptográfico que permite verificar la autenticidad, integridad y el no repudio de un documento o mensaje electrónico.

Gestión de claves criptográficas: Conjunto de políticas, procedimientos y mecanismos destinados a administrar el ciclo de vida de las claves criptográficas.

Hardware Security Module (HSM): Dispositivo físico especializado diseñado para proteger y gestionar de forma segura claves criptográficas y realizar operaciones criptográficas críticas dentro de un entorno protegido.

HSM por software: Implementación de funcionalidades equivalentes a un *HSM* mediante componentes software, sin requerir hardware criptográfico especializado.

Integridad: Propiedad de seguridad que garantiza que los datos no han sido alterados de forma no autorizada.

Interoperabilidad: Capacidad de diferentes sistemas o aplicaciones para intercambiar información y utilizarla de manera efectiva.

Key Encryption Key (KEK): Clave criptográfica utilizada para proteger otras claves criptográficas mediante técnicas de cifrado o envoltura de claves.

Módulo criptográfico: Componente hardware o software encargado de ejecutar operaciones criptográficas y proteger material criptográfico sensible.

No repudio: Propiedad de seguridad que impide que una entidad niegue posteriormente la autoría de una acción o transacción realizada.

Proveedor de identidad: Servicio responsable de autenticar usuarios y emitir credenciales o tokens que pueden ser utilizados por aplicaciones y servicios para verificar identidades.

Rol: Conjunto de permisos y responsabilidades asignados a una categoría de usuarios dentro de un sistema.

Rotación de claves: Proceso de sustitución periódica de una clave criptográfica por una nueva para reducir riesgos de compromiso.

Semántica KMIP: Conjunto de conceptos, operaciones y estructuras de gestión de claves inspirados en el estándar *Key Management Interoperability Protocol*, independientemente de la implementación completa del protocolo.

Token: Credencial o artefacto digital utilizado para representar información de identidad, autenticación, autorización o sesión, que permite a una entidad demostrar determinados privilegios o atributos ante un sistema.

Trazabilidad: Capacidad de reconstruir y verificar la secuencia de eventos y acciones realizadas dentro de un sistema.

Verificación de firma: Proceso criptográfico mediante el cual se comprueba la validez de una firma digital mediante el uso de la clave pública correspondiente.

Zeroización: Técnica de seguridad que consiste en sobrescribir o eliminar de forma controlada datos sensibles almacenados en memoria o medios de almacenamiento para impedir su recuperación.